COMPLETE STUDY GUIDE · DAYS 1–15

Learning Blockchain in 15 Days

From a single SHA-256 hash to a full-stack certificate dApp: hashing, Proof of Work, signatures, Merkle trees, the EVM, Solidity, tokens, NFTs, security, DeFi, gas, upgrades, oracles, Layer 2, zero-knowledge and account abstraction.



Manish Khadka
Full Stack Developer · Team Lead
khadka-manish.com.np · github.com/Khadka1996

15 – 29 October 2026
github.com/Khadka1996/Learning-Blockchain

Contents

	About this guide & the journey at a glance	3
01	Hashing & SHA-256	4
02	Blocks, chains & Proof of Work	10
03	Keys, signatures & transactions	16
04	Merkle trees, forks & consensus	22
05	Ethereum & the EVM	28
06	Solidity fundamentals	33
07	ERC-20 tokens	39
08	NFTs: ERC-721 & ERC-1155	45
09	Smart-contract security	50
10	DeFi: AMM from scratch	55
11	Gas optimisation & upgradeable contracts	59
12	Oracles, Layer 2 & bridges	64
13	Full-stack dApp	69
14	Zero-knowledge & account abstraction	74
15	Capstone: certificate dApp + self-audit	78
	What I learned: 5 lessons, glossary & next steps	82

Page numbers refer to the page counter of your PDF reader. Each chapter also has its own "Page X of Y" footer. The bookmarks panel lets you jump straight to any day.

How each chapter is organised

Every chapter follows the same pattern, so it is easy to revise: **learning objectives** → **the concept, step by step** (in plain words, with diagrams) → **key code walkthrough** → **real results from my run** → **QA & security findings** (what broke and how it was fixed) → **common mistakes and interview questions** → **exercises** → **cheat sheet** → **references**. All code, tests and full QA reports are in the GitHub repository, one folder per day.

How to use it

- **First read:** go in order. Every day builds on the ones before it (the hash from Day 1 is inside almost everything that follows).
- **Revision:** read only the cheat-sheet boxes and the interview questions of each day.
- **Practice:** try the exercises before looking at the solutions in each day's `exercises/solutions` folder.

About this guide

For 15 days I learned blockchain in public, one topic a day, going from the very basics to advanced topics. Each morning I learned the concept and built working code; each evening I tested it like an attacker, fixed what broke, wrote exercises and a study guide. This book puts all 15 study guides together in one place, in order, so the whole journey can be read from start to finish.

15
topics, basic → advanced

15
working projects with tests

15
QA & security reports

45
exercises (easy / medium / challenge)

The journey at a glance

Day	Topic	What I built
PHASE 1 — FOUNDATIONS		
01	Hashing & SHA-256	A tested hash library, avalanche-effect statistics, a brute-force demo
02	Blocks, chains & Proof of Work	A mineable blockchain with tamper detection
03	Keys, signatures & transactions	secp256k1 wallets, signed transfers, a replay-safe ledger
04	Merkle trees, forks & consensus	Merkle proofs, the most-work fork rule, a P2P fork simulation
PHASE 2 — ETHEREUM & SOLIDITY		
05	Ethereum & the EVM	A mini-EVM whose gas matches Hardhat step for step; storage layout explorer
06	Solidity fundamentals	A tested ETH vault: events, modifiers, custom errors, a re-entrancy attack and fix
07	ERC-20 tokens	A token from scratch and the OpenZeppelin version; a vault that survives odd tokens
08	NFTs: ERC-721 & ERC-1155	An NFT from scratch, a paid collection with royalties, ERC-1155 game items
PHASE 3 — ADVANCED		
09	Smart-contract security	5 broken contracts, a working exploit and a fix for each
10	DeFi: AMM from scratch	An x·y=k swap pool with fees and LP shares; a sandwich bot vs slippage limits
11	Gas optimisation & upgrades	The same contract with up to 70% less gas; a UUPS proxy upgraded V1 → V2
12	Oracles, Layer 2 & bridges	A safe price reader, a simulated optimistic rollup with fraud proofs, a safe bridge
13	Full-stack dApp	TipJar: contract + web page with ethers.js and MetaMask, browser tests
14	Zero-knowledge & account abstraction	A circom/snarkjs proof checked on-chain; an ERC-4337 smart wallet with a daily limit
PHASE 4 — CAPSTONE		
15	Certificate dApp + self-audit	CertCheck: issue, revoke and verify certificates by fingerprint, plus a professional self-audit

LEARNING BLOCKCHAIN · STUDY GUIDE

Day 01 — Hashing & SHA-256

Date: Thursday 15 October 2026 · **Phase:** Foundations (Day 1 of 15) · **Author:** Manish Khadka
Code: github.com/Khadka1996/Learning-Blockchain/tree/main/day-01 · 33 tests passing

Learning objectives

After this day you should be able to:

1. Explain in plain words what a cryptographic hash function is and name its six key properties.
2. Compute SHA-256 of text, bytes and files in Node.js and check results against official NIST test vectors.
3. Measure the **avalanche effect** (≈ 50 % of output bits flip for a one-character change).
4. Explain why hashes are one-way, and how brute-forcing "leading zeros" becomes **Proof of Work**.
5. Spot the input-handling bugs that break hashing code in practice: number parsing, malformed Unicode, file races.

The concept, step by step

Step 1 — A fingerprint for data

A hash function takes **any** input — a word, a payment, a 2 GB file — and returns a short, fixed-size output called a *digest*. SHA-256 always returns 256 bits, written as 64 hexadecimal characters. Like a fingerprint, the digest is tiny compared with the original, belongs to exactly one input in practice, and cannot be used to rebuild the input.



Figure 1 — any input, one fixed-size fingerprint. There is no path back from the digest to the input.

Step 2 — The six properties that blockchains rely on

Property	Meaning	Why a blockchain needs it
Deterministic	Same input → same output, always	Thousands of computers can check the same data independently
Fixed size	Always 256 bits	A block can point to the previous block with a 32-byte reference
Fast	Cheap to compute	Verifying is quick for everyone
Avalanche effect	Tiny change → ~50 % of bits flip	Tampering is instantly visible

Preimage resistant	Cannot work back from digest to input	Enables Proof of Work and commitments
Collision resistant	Infeasible to find two inputs with one digest	The digest can safely stand in for the data

Step 3 — The avalanche effect, measured

Changing `"Manish sends 10 coins"` to `"Manish sends 11 coins"` — one digit — changed **137 of 256 bits (53.5 %)**. Repeating the experiment 10,000 times with random messages gave an average of **127.93 bits (49.97 %)**, with a minimum of 99 and a maximum of 156. Each output bit behaves like a fair coin toss, so theory predicts 128 ± 8.

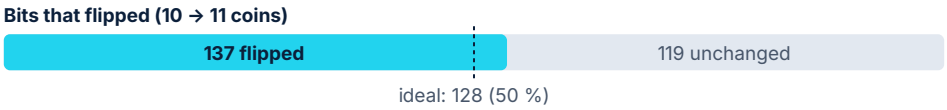


Figure 2 — one edited character changes more than half the fingerprint.

Step 4 — One-way, so the only strategy is guessing

To find an input whose hash starts with *n* zero hex digits, there is no formula: you try input after input. Each hex digit has 16 possible values, so each extra zero makes the search about **16× harder** (expected 16^n attempts). Bitcoin mining is this same search at enormous scale, and the "difficulty" is simply how many zeros (more precisely, how small a number) the hash must be.

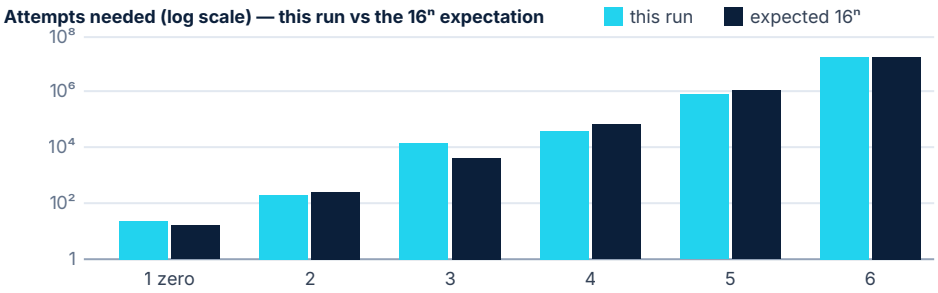


Figure 3 — each extra zero multiplies the work by ~16. Single runs vary (luck), the trend never does.

Step 5 — Why this builds a chain (preview of Day 02)

Each block stores the hash of the block before it. Edit any old block and its hash changes, which breaks the pointer in the next block, which breaks the next one, and so on. Combine that with Proof of Work and rewriting history means redoing all the guessing for every later block.



Figure 4 — every block carries the previous block's fingerprint (illustrative values).

Key code walkthrough

All logic lives in `src/lib.js` as small, pure, validated functions; the CLI scripts only parse arguments and print. The snippets below are the real code after the evening audit.

1. Hashing — with input checks that prevent silent collisions

```
// Only well-formed strings, Buffers and Uint8Arrays are accepted.
function assertHashable(input) {
  if (typeof input !== "string" && !(input instanceof Uint8Array)) {
    throw new TypeError(`sha256 expects a string, Buffer or Uint8Array, …`);
  }
  // A lone UTF-16 surrogate has no UTF-8 form; Node would silently turn it into U+FFFD,
  // so "\uD800", "\uDC00" and "\uFFFD" would all share one hash (audit finding E1).
  if (typeof input === "string" && !isWellFormed(input)) {
    throw new TypeError("sha256 expects a well-formed Unicode string …");
  }
}
function sha256Bytes(input) {
  assertHashable(input);
  return createHash("sha256").update(input).digest();   // 32-byte Buffer
}
```

2. Measuring the avalanche — counting differing bits

```
function bitsDifferent(a, b) {             // Hamming distance of two 32-byte digests
  let diff = 0;
  for (let i = 0; i < a.length; i++) {
    let x = a[i] ^ b[i];                  // XOR: 1 wherever the bits differ
    while (x) { diff += x & 1; x >>>= 1; }  // count the 1s in this byte
  }
  return diff;                             // 0 … 256
}
```

3. Brute force — the idea behind Proof of Work

```
function findLeadingZeros(zeros, { prefix = "manish-", maxAttempts = 2 ** 32 } = {}) {
  const z = parseZeros(zeros);             // whole number 1–8 only (F1–F3, E4)
  if (typeof prefix !== "string") throw new TypeError("prefix must be a string");  // E5
  const target = "0".repeat(z);
  for (let nonce = 0; nonce < maxAttempts; nonce++) {   // always terminates (F3)
    const hash = sha256Hex(`${prefix}${nonce}`);
    if (hash.startsWith(target)) return { found: true, nonce, hash, attempts: nonce + 1 };
  }
  return { found: false, attempts: maxAttempts };
}
```

4. File fingerprints — check the file you actually read

```
// Open ONCE, then check the open handle. Checking the path and re-opening it left a
// gap where the file could be swapped for a pipe that never ends (audit finding E2).
const handle = await fs.promises.open(path, O_RDONLY | O_NONBLOCK);
const info = await handle.stat();
if (!info.isFile()) { await handle.close(); throw new Error(`not a regular file: ${path}`); }
handle.createReadStream()
  .on("data", (chunk) => { bytes += chunk.length; hash.update(chunk); })  // size = bytes read
  .on("end",  () => resolve({ bytes, hex: hash.digest("hex") }));
```

Results from the run

Experiment	Result
"10 coins" → "11 coins"	137 / 256 bits (53.5 %)
10,000 random 1-char edits	avg 127.93 (49.97 %)
Fewest / most bits flipped	99 / 156
Bitcoin genesis id (exercise)	10 leading zeros
Tests · lint · audit	33 pass · clean · 0 vulns

Zeros	Attempts (run)	Expected 16^n
1	23	16
2	193	256
3	13,699	4,096
4	37,213	65,536
5	742,576	1,048,576
6	16,325,200	16,777,216

The difficulty table is now generated by `npm run preimage:table`, and a test pins the first rows, so the notes and the code cannot drift apart. Five zeros took 742,576 guesses (about 2 seconds on a laptop); six took 16.3 million (≈ 44 s).

QA & security findings

The morning review found and fixed 8 issues (F1–F9; F7 accepted as documented behaviour). The evening audit — run as an outside reviewer — found 5 more. Every one got a failing test first, then the fix.

ID	Severity	Problem	Fix
F1	**Medium**	`preimage.js abc` → `NaN` zeros → every hash "matched" — a fake success.	Strict validation, exit code 1
F3	**Medium**	65 zeros (more than a hash has) → infinite loop.	Cap at 8 + `maxAttempts`
F8	**Medium**	Fingerprinting `/dev/zero` streamed forever (denial of service).	Regular files only
E1	**Medium**	**Real collision:** `"\uD800"`, `"\uDC00"` and `"\uFFFD"` all hashed to `83d544cc…` because Node replaces lone surrogates before hashing.	Reject malformed strings; 5,000-case fuzz test
E2	**Low**	Time-of-check / time-of-use race: path checked, then re-opened.	Open once, check the handle
E3	**Low**	`hash.js ""` hashed the demo text instead of the empty string.	Default only when no argument
E4	**Low**	`--stats 0x10` ran 16 trials; `parseZeros([5])` was accepted.	One strict `toWholeNumber()`
E5	**Info**	Non-string `prefix` became `"[object Object]"`.	`TypeError`

Known issues (accepted): F7 — visually identical Unicode text can hash differently, so normalise user text with `normalize("NFC")`; E6 — invalid UTF-8 in command-line arguments is replaced by

Node before our code runs; E7 — 8 zeros is allowed and takes hours by design. Full details: `day-01/QA.md`.

Common mistakes

- **Hashing the hex text instead of the bytes.** `sha256("ab")` and `sha256(Buffer.from("ab","hex"))` are different inputs. Bitcoin hashes raw bytes.
- **Forgetting byte order.** Bitcoin shows block and transaction ids byte-reversed; compare like with like.
- **Assuming strings are bytes.** Text must be encoded (UTF-8) first; malformed or un-normalised text gives surprising hashes (E1, F7).
- **Trusting `Number()` on user input.** It happily accepts `"0x10"`, `"1e3"` and `" "` (→ 0).
- **Hashing without a salt** when the input space is small (votes, moves, PINs): an attacker just hashes every option.
- **Using a plain hash as a password store or a MAC.** Use a slow password hash (scrypt, Argon2) or HMAC instead.

Five interview questions

1. What is the difference between preimage resistance and collision resistance?

Preimage resistance: given a digest, you cannot find *any* input that produces it (≈ 2^{256} work for SHA-256). Collision resistance: you cannot find *any two* inputs with the same digest (≈ 2^{128} work, because of the birthday paradox). Collisions are easier because the attacker chooses both inputs.

2. Why does Bitcoin hash block headers twice (double SHA-256)?

Satoshi never documented the reason; the common explanation is defence-in-depth against length-extension and other weaknesses of the Merkle–Damgård construction. In practice it gives the `hash256` function used for block and transaction ids.

3. How does Proof of Work use a hash function?

Miners change a nonce in the block header until `hash256(header)` is below a target. Because the hash is one-way and unpredictable, the only method is guessing; the expected number of guesses is set by the difficulty, and anyone can verify the result with one hash.

4. A teammate hashes user names to create unique ids. What can go wrong?

Visually identical names can have different bytes (Unicode normalisation, F7), and malformed strings can collapse into the same bytes (E1). Normalise to NFC, reject malformed text, and hash a canonical byte encoding.

5. Why is `sha256(secret ‖ message)` not a safe way to authenticate a message?

SHA-256 is vulnerable to length extension: knowing the digest and the length, an attacker can compute the hash of `secret ‖ message ‖ padding ‖ extra` without the secret. Use HMAC-SHA-256 (or SHA-3, which is not affected).

Exercises (in `day-01/exercises/`)

Level	Exercise	Goal
Easy	Spot the tampered record	Re-hash each record and return the ids whose text no longer matches its published hash. `npm run ex:easy`
Medium	Verify Bitcoin's genesis block	Implement double SHA-256 and reproduce block 0's id `000000000019d668…` (10 leading zeros) from its 80-byte header. `npm run ex:medium`
Challenge	Commit–reveal	Build salted commitments for rock–paper–scissors, show the 3-guess attack on unsalted ones, verify with a constant-time compare. `npm run ex:challenge`

Cheat sheet — Day 01

SHA-256 output	256 bits = 32 bytes = 64 hex characters
Node.js	`crypto.createHash("sha256").update(data).digest("hex")`
Empty-string hash	`e3b0c442…7852b855` (a quick sanity check)
Avalanche	1 changed bit/char → ~128 of 256 bits flip
Brute force n hex zeros	$\approx 16^n$ attempts on average
Security levels	preimage $\approx 2^{256}$ · collision $\approx 2^{128}$
Bitcoin ids	`hash256 = SHA-256(SHA-256(x))`, shown byte-reversed
Text input	encode UTF-8 · reject lone surrogates · `normalize("NFC")`
Secrets	HMAC for messages · salt for small choices · scrypt/Argon2 for passwords

References

1. NIST FIPS 180-4, *Secure Hash Standard (SHS)* — https://csrc.nist.gov/pubs/fips/180-4/upd1/final
2. NIST, *Cryptographic Standards and Guidelines — Example Values (SHA-256 test vectors)* — https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines/example-values
3. S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (2008) — https://bitcoin.org/bitcoin.pdf
4. A. Antonopoulos & D. Harding, *Mastering Bitcoin*, 3rd ed. (open edition) — https://github.com/bitcoinbook/bitcoinbook
5. P. Rogaway & T. Shrimpton, *Cryptographic Hash-Function Basics*, FSE 2004 — https://web.cs.ucdavis.edu/~rogaway/papers/relates.pdf
6. Bitcoin Developer Guide — Block Chain — https://developer.bitcoin.org/devguide/block_chain.html
7. ethereum.org — Proof-of-work (PoW) — https://ethereum.org/en/developers/docs/consensus-mechanisms/pow/
8. MDN — `String.prototype.isWellFormed()` (lone surrogates) — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/isWellFormed
9. MITRE CWE-367, *Time-of-check Time-of-use (TOCTOU) Race Condition* — https://cwe.mitre.org/data/definitions/367.html

PHASE 1 · FOUNDATIONS

Day 02 — Blocks, chains & Proof of Work

Learning Blockchain in 15 Days · Day 2 of 15 · Friday 16 October 2026 · Manish Khadka

Yesterday's hash becomes today's glue: put data in a block, link every block to the hash of the one before it, and make each block expensive to create. That is how a ledger becomes tamper-evident and tamper-resistant.

Learning objectives

1. Explain what a block header contains and why each block stores the previous block's hash.
2. Explain Proof of Work: what the nonce and the difficulty are, and why the expected work is about 2^d hashes.
3. Build a chain validator that reports *where* and *why* a chain breaks, and list the rules it must enforce.
4. Show, with real numbers, why rewriting history means re-mining every later block.
5. Recognise the classic bugs: unchecked targets, self-declared difficulty, ambiguous encodings, non-canonical JSON.

The concept, step by step

Step 1: a block is a box with a label

The box holds data (here: a payment). The label is the SHA-256 hash of the block's **header**. Change anything inside the box and the label changes completely (Day 01's avalanche effect).

Step 2: every box holds the previous box's label

Each header contains `prevHash`, the hash of the block before it. That makes a chain: block 2 depends on block 1, which depends on block 0, the fixed **genesis block**.



Figure 1 — Real blocks from today's run (16-bit difficulty). Each block's hash becomes the next block's prevHash.

Step 3: labels are hard to make (Proof of Work)

A block is accepted only if its hash, read as a 256-bit number, is below a **target** 2^{256-d}, in other words it starts with **d zero bits**. Nobody can predict which header gives such a hash, so the miner changes a counter (the **nonce**) and hashes again, over and over. Each try succeeds with probability $1/2^d$, so a block takes about **2^d guesses** on average, while checking it takes **one** hash.

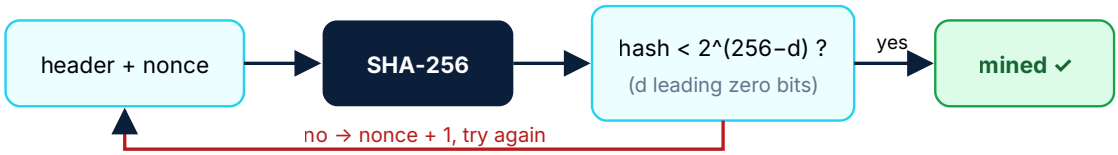


Figure 2 — The mining loop. The only strategy is guessing.

Step 4: why the past becomes (practically) fixed

Edit block 1 and its hash changes, so it no longer meets the target, and block 2's `prevHash` no longer matches. To hide the edit an attacker must re-mine block 1 *and every block after it*, while honest miners keep extending the real chain. The more blocks on top (**confirmations**), the harder it gets. Satoshi's whitepaper (§11) shows the attacker's chance falls exponentially: with 10% of the hash power and 6 blocks behind it is about 0.02%.

Key code walkthrough

1. Unambiguous header encoding (`src/block.js`)

```
// Every field has a fixed type and its own slot: two different headers never give the same bytes.
function serializeHeader(b, dataHash = hashData(b.data)) {
  return JSON.stringify([HEADER_VERSION, b.index, b.prevHash, b.timestamp,
                         b.difficulty, b.nonce, dataHash]);
}
// v1 (tutorial style) — BROKEN: index 1 + timestamp 12 reads exactly like index 11 + timestamp 2
sha256(b.index + b.prevHash + b.timestamp + JSON.stringify(b.data) + b.nonce + b.difficulty)
```

2. Mining with a fast zero-bit check

```
const json = encodeData(data);             // read the caller's payload ONCE (audit E2)
const dataHash = sha256Hex(json);          // payload is fixed while mining: hash it once
header.data = deepFreeze(JSON.parse(json)); // stored data = exactly the bytes that were hashed

for (let nonce = 0; nonce < maxAttempts; nonce++) {  // always terminates (F8)
  header.nonce = nonce;
  const digest = sha256(serializeHeader(header, dataHash));
  if (hasLeadingZeroBits(digest, difficulty)) {
    const block = Object.freeze({ ...header, hash: digest.toString("hex") });
    return { block, attempts: nonce + 1 };
  }
}
```

3. The validator is the security (`src/chain.js`)

```
function checkBlock(block, parent, { difficulty, now }) {
  if (block.hash !== hashBlock(block))           return "block was edited";
  // F2: the RULES decide the difficulty, never the block itself
  if (block.difficulty !== difficulty)           return "wrong difficulty";
  // F1: anyone can recompute a hash; only the target proves work
  if (!meetsDifficulty(block.hash, difficulty))  return "not mined";
  if (parent === null) return null;            // genesis is compared separately (F4)
  if (block.index !== parent.index + 1)          return "wrong index";     // F6
  if (block.prevHash !== parent.hash)            return "chain broken";
  if (block.timestamp < parent.timestamp)        return "earlier than parent";
  if (block.timestamp > now + MAX_FUTURE_MS)     return "over 2 h in the future";
  return null;
}
```

4. Evening improvement: difficulty adjustment

```
// First block of each new window: compare how long the last window took with the target.
const actual = Math.max(1, blocks[height - 1].timestamp - blocks[first].timestamp);
const wanted = gaps * retarget.targetMs;
// ±1 bit = ×2 work; at most ±2 bits (×4) per step, like Bitcoin
const step = clamp(Math.round(Math.log2(wanted / actual)), -2, +2);
return clamp(parentDifficulty + step, 1, 32);
```

Results from the run

A 5-block chain at 16 bits (`npm start`): every hash starts with `0000`, and every prevHash equals the hash above it.

```
#  data                      nonce   hash (first 16 hex)  prevHash
0  Day 02 genesis — Learning 145549  0000d39b694b706e     0000000000000000
1  Sita pays Ram 5 coins      36720  0000ae20c4515493     0000d39b694b706e
2  Ram pays Hari 2 coins      21787  0000059caabc359e     0000ae20c4515493
3  Hari pays Gita 1 coin      96206  0000ae62be3fc208     0000059caabc359e
4  Gita pays Sita 3 coins      5882  0000acbcb8c34e77     0000ae62be3fc208
Mined 4 blocks: 160,599 hashes in 468 ms · Chain valid: true
```

Tampering (`npm run tamper`), changing "5 coins" to "500 coins" in block 1:

```
1. Honest chain                  → VALID
2. Edit block 1                  → INVALID at block 1: hash does not match contents
3. Recompute its hash (no mining) → INVALID at block 1: does not meet the PoW target
4. Re-mine block 1 only          → INVALID at block 2: prevHash does not match
5. Re-mine blocks 1–4            → VALID
Rewriting history cost 491,216 hashes (4 blocks re-mined).
```

Difficulty vs. work (8 fixed samples per level):

Bits	Avg attempts	Expected 2^d	Avg time
4	17	16	0.24 ms
8	262	256	1 ms
12	3,580	4,096	12 ms
16	46,606	65,536	143 ms
20	905,326	1,048,576	2,910 ms

Difficulty adjustment (target 100 ms/block, retarget every 5 blocks):

Blocks	Bits	Avg time
1–5	8	2.1 ms
11–15	12	18.8 ms
21–25	16	163.9 ms
26–30	15	66.6 ms
36–40	16	114.7 ms

Each +4 bits ≈ 16× more work. Attempt counts are identical on every machine; times depend on the CPU (~330,000 hashes/s here). Quality: **40 / 40 tests pass**, ESLint clean, `npm audit` 0 vulnerabilities.

QA & security findings

The first draft followed the common tutorial pattern. An attack script against it, then an independent evening audit, found 14 issues. Each got a failing test first, then a fix.

ID	Severity	What was wrong	Fix
F1	**High**	Validator never checked the target: edited blocks with recomputed, unmined hashes passed.	Require `meetsDifficulty` for every block.
F2	**High**	Difficulty read from the block itself: a forger wrote `difficulty: 1`.	Difficulty comes from the chain's rules.
F3	**Medium**	String concatenation: two different headers, one hash.	Typed JSON-array encoding.
F4	**Medium**	Genesis never checked: whole fake histories accepted.	Block 0 must equal the rule-defined genesis.
F6	**Medium**	Index 99 after 1, timestamps far in the past or future accepted.	index = parent + 1; parent ≤ time ≤ now + 2 h.
F8	**Medium**	Difficulty 256 → infinite mining loop (DoS).	1–32 bits; `maxAttempts`.
E1	**Medium**	`JSON.stringify` merged payloads: `{a: undefined} = {}`, `NaN = null`; a `toJSON()` showing 500 coins hashed as 5.	Strict canonical encoder (sorted keys, plain JSON only).
E2	**Medium**	Chain stored the caller's object: editing your variable rewrote the ledger.	Deep-frozen copy of the exact hashed bytes.
E3–E6	**Low**	Unhashed extra fields accepted; `addBlock` appended invalid blocks; no payload size limit; a bad clock value switched off the future rule.	Exact field set; validate before append; 1 MB limit; strict `now`.
F5, F7	**Low / Info**	Malformed headers hashed as "undefined"; raw BigInt error.	Shape checks; clear errors.

Known issue K1 (low): with difficulty adjustment, miners can fake slow timestamps (time-warp) to lower difficulty, bounded here to −2 bits per window and the 2-hour rule. Bitcoin mitigates this with median-time-past.

Common mistakes

- **Checking "hash = recomputed hash" only.** Anyone can recompute a hash. The *target* is what proves work.
- **Trusting what a block says about itself** (its difficulty, its genesis). Rules come from the protocol, never from the data being checked.
- **Gluing fields together as text** or hashing `JSON.stringify` output. Hash canonical, unambiguous bytes.
- **Validating only from block 1**, or only links and not indexes and timestamps.
- **Thinking Proof of Work makes edits impossible.** It makes them *expensive*; security comes from honest miners holding most of the hash power.

Interview questions

1. Why does editing one old block invalidate every block after it?

The edited block's hash changes (avalanche effect). The next block stores the old hash in `prevHash`, so the link breaks; fixing it changes that block's hash too, and so on to the tip. Each repaired block must also be re-mined.

2. What is "difficulty", and how much work does a block take?

The hash must be below a target 2^{256-d} (d leading zero bits). Each guess succeeds with probability 2^{-d}, so the expected work is 2^d hashes. Today: 16 bits ≈ 65,536 hashes; measured average 46,606 over 8 blocks.

3. Why is Proof of Work "asymmetric"?

Finding a valid nonce needs about 2^d hashes; verifying it needs one. So every node can cheaply check work that was expensive to produce. This idea goes back to Dwork & Naor (1992) and Hashcash (2002).

4. Why must the validator compute the required difficulty itself?

If it trusted the block's own difficulty field, an attacker could declare difficulty 1 and re-mine history almost for free (finding F2). Bitcoin re-derives the target from the previous 2016 blocks' timestamps.

5. Why do exchanges wait for about 6 confirmations?

From Satoshi's §11 formula: an attacker with 10% of the hash power catches up from z = 6 blocks behind with probability ≈ 0.024%, and from z = 1 with ≈ 20%. More confirmations make reversal exponentially less likely.

Exercises summary

Level	Exercise	Key idea
Easy	Where does the chain break? `findBreak(blocks)` returns the first bad block and how many must be re-mined.	Everything after an edit depends on it.
Medium	Implement Satoshi's attacker-success formula; reproduce the whitepaper's tables (q = 0.1, z = 5 → 0.0009137).	Why 6 confirmations.
Challenge	Decode Bitcoin's compact `bits` target (`0x1d00ffff`) and verify the genesis block's PoW with double SHA-256.	Real Bitcoin rules: hash ≤ target, little-endian.

Cheat sheet

- **Block hash** = SHA-256(header); header = version, index, prevHash, time, difficulty, nonce, dataHash
- **Valid PoW:** hash < 2^{256-d} ⇔ d leading zero bits
- **Expected work:** 2^d hashes; +1 bit = ×2, +4 bits = ×16
- **Verify cost:** 1 hash
- **Validator checks:** our genesis · hash matches · difficulty from rules · target met · index +1 · prevHash link · time ≥ parent, ≤ now + 2 h
- **Bitcoin:** double SHA-256, compact `bits` target, retarget every 2016 blocks (×4 max), 10-min blocks
- **Run:** `npm start · npm run tamper · npm test`

References

1. **Bitcoin: A Peer-to-Peer Electronic Cash System**, S. Nakamoto (2008), §3, §4, §11 — https://bitcoin.org/bitcoin.pdf
2. **Bitcoin Developer Guide: Block Chain** — https://developer.bitcoin.org/devguide/block_chain.html
3. **Bitcoin Developer Reference: Block Headers & Target nBits** — https://developer.bitcoin.org/reference/block_chain.html
4. **Mastering Bitcoin, 3rd ed.**, A. Antonopoulos & D. Harding, ch. 11–12 — https://github.com/bitcoinbook/bitcoinbook
5. **Pricing via Processing or Combatting Junk Mail**, C. Dwork & M. Naor (CRYPTO 1992) — https://www.wisdom.weizmann.ac.il/~naor/PAPERS/pvp.pdf
6. **Hashcash: A Denial of Service Counter-Measure**, A. Back (2002) — http://www.hashcash.org/papers/hashcash.pdf
7. **ethereum.org: Proof-of-Work** — https://ethereum.org/en/developers/docs/consensus-mechanisms/pow/
8. **RFC 8785: JSON Canonicalization Scheme** — https://www.rfc-editor.org/rfc/rfc8785
9. **NIST FIPS 180-4: Secure Hash Standard** — https://csrc.nist.gov/pubs/fips/180-4/upd1/final

Code, tests and the full QA report: https://github.com/Khadka1996/Learning-Blockchain/tree/main/day-02

PHASE 1 · FOUNDATIONS

Day 03 — Keys, signatures & transactions

Learning Blockchain in 15 Days · Day 3 of 15 · Saturday 17 October 2026 · Manish Khadka

Day 02's chain let anyone write "Sita pays Ram 5 coins". Today only Sita can: every payment carries a digital signature that only her private key can make and everyone can check with her public key.

Learning objectives

1. Explain the chain **private key → public key → address**, and why it only works in one direction.
2. Explain what an ECDSA signature (r, s) proves, and what it does *not* prove on its own.
3. List the fields a transaction must sign (domain, chain id, from, to, amount, nonce) and the attack each field blocks.
4. Recognise the classic signature bugs: replay, cross-chain replay, malleability (high-S), time-of-check/time-of-use, message-type confusion.
5. Use checksummed addresses to catch typos before coins are lost.

The concept, step by step

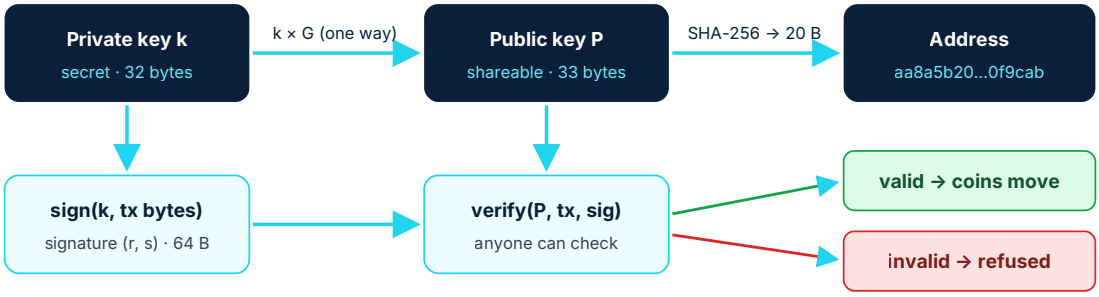


Figure 1 — The whole day in one picture. Only the holder of k can make a signature that P accepts.

Step 1: a private key is a secret number

On **secp256k1**, the curve Bitcoin and Ethereum use, a private key is any whole number from 1 to $n - 1$, with $n \approx 1.16 \times 10^{77}$. Like a PIN, except nobody can guess it: with about 10^{77} possible keys, trying them one by one is hopeless for any computer that will ever exist.

Step 2: the public key is computed from it, one way

`P = k × G`: "add" the curve's starting point *G* to itself *k* times (with a fast double-and-add trick). Going forward takes microseconds; going back, finding *k* from *P*, is the **discrete logarithm problem**. Like mixing paint: easy to mix, impossible to un-mix.

Step 3: the address is a short fingerprint of the public key

This repo uses the first 20 bytes of SHA-256(public key). Bitcoin uses RIPEMD-160(SHA-256(key)); Ethereum the last 20 bytes of Keccak-256. Same idea: an account number you can share.

Step 4: a signature is a stamp tied to the secret *and* the exact message

The signer feeds the private key and the exact bytes into ECDSA and gets two numbers (r, s). Anyone with the public key can check them. Change one bit of the message, or use another key, and the check fails.

Step 5: a transaction is a signed message, and every field earns its place

A valid signature only proves "the owner of this key signed these bytes". It does not say *when*, *how many times* or *on which network*. That is the job of the other signed fields:

Field	Plain meaning	Attack it blocks
`domain`	"this is a transfer"	Reusing a note or login signature as a payment
`chainId`	which network	Cross-chain replay (testnet → mainnet), Ethereum's EIP-155
`from`	sender's public key	Signing "as Sita" with another key
`to`, `amount`	who gets how many whole coins	Editing or redirecting after signing; rounding tricks
`nonce`	"my payment number 0, 1, 2..."	Replay: the same payment applied twice

Step 6: one payment, one signature (low-S)

For any valid ECDSA signature (r, s), the pair (r, *n* − s) is also valid. Anyone can make this "twin" without the key, which changes anything that hashes the signature. Bitcoin (BIP 62/146) and Ethereum (EIP-2) force **s ≤ n/2**; this repo does the same on both sign and verify.

Key code walkthrough

The exact bytes that get signed — `src/transaction.js`

```
function signingBytes(tx) {
  assertBody(tx);  // version, chainId, key, 40-hex "to", safe whole amount ≥ 1, nonce ≥ 0
  return Buffer.from(JSON.stringify(
    ["learning-blockchain/transfer", tx.version, tx.chainId, tx.from, tx.to, tx.amount,
tx.nonce]));
}
// tx id = SHA-256(signingBytes): the signature is NOT part of the id
```

A fixed-order JSON *array* of already-validated primitives: no key order, no `toJSON`, no surprises (Day 02's lesson).

Low-S signing — `src/keys.js`

```
const sig = crypto.sign("sha256", messageBytes(message), { key, dsaEncoding: "ieee-p1363" });
let s = toBig(sig.subarray(32));
if (s > HALF_N) s = N - s;          // always the lower twin
// verify refuses r = 0, r ≥ n, s = 0 and s > n/2 before any maths
```

Read once, then check, then use — `verifyTransfer`

```
tx = snapshot(input);        // every field read exactly ONCE into a frozen copy (audit M1)
bytes = signingBytes(tx);
if (!verifySignature(tx.from, bytes, tx.signature)) return { ok: false, … };
return { ok: true, id, sender: addressFromPublicKey(tx.from), tx };  // callers use THIS tx
```

The ledger after the evening audit — `src/ledger.js`

```
class Ledger {
  #chainId; #totalSupply; #balances = new Map(); #nonces = new Map();   // private (E1)
  get chainId() { return this.#chainId; }                          // read-only
  applyAll(txs) {                                                    // all-or-nothing
    for (let i = 0; i < length; i++)
      if (!Object.hasOwn(txs, i)) throw new TypeError(`transaction ${i} is missing`); // E2
    … check every tx on a scratch copy, then commit once
  }
}
Object.freeze(Ledger.prototype);
```

Evening improvement: checksummed addresses

```
const h = sha256(address).toString("hex");       // hash of the lowercase address
for (let i = 0; i < 40; i++)
  out += parseInt(h[i], 16) >= 8 ? address[i].toUpperCase() : address[i];  // EIP-55 idea
```

Results from the run

```
$ npm start
Sita  address aa8a5b204a46…0f9cab
Ram   address 6e0a062d96a1…c7d57b
Hari  address 0007bbbef172…47087e
Sita → Ram 30, Ram → Hari 10
Sita 70 · Ram 20 · Hari 10
total 100 (supply 100)
```

```
$ npm run typo
AA8A5b204a46BE63Be0862C239aC0911E70f9CAB
typos tried:  10,000
caught:        9,999 (99.99%)
missed:            1

$ npm run bench   (2,000 payments)
sign 0.384 ms · verify 0.447 ms each
```

```
$ npm run attack    (13 tricks: forged key, edited amount, redirect, replay twice,
                    cross-chain replay, high-S twin, overdraft, 0.1 coins, 2^53+1 coins,
                    "__proto__" address, half-applied batch, garbage key, lying getter)
Mallory tried 13 attacks: 13 refused, 0 got through.

$ npm test           # tests 38 · pass 38 · fail 0
$ npm run lint       eslint: 0 problems
$ npm audit --omit=dev   found 0 vulnerabilities
```

Signatures and timings change per run (ECDSA uses a fresh random number each time; speed depends on the machine). Addresses are fixed because the demo keys are derived from names — never do that with real money.

QA & security findings

Morning (attack script + manual review): 10 fixed — **2 high** (B3 replay: Ram received 40 coins twice; M1 a getter read 1 coin when checked and 100 when paid), **6 medium** (high-S twin, fractions, half-applied batch, cross-chain replay, precision loss above 2^{53}, `__proto__` address), **2 low** (any string as recipient, crash on garbage key).

Evening (independent audit, failing test first):

ID	Severity	What was wrong	Fix
E1	**Medium**	Ledger state was public: `ledger.balances.set(mallory, 1e6)` minted a million coins; `ledger.chainId = "lb-testnet"` switched off the cross-chain check.	Private `#fields`, read-only getters, frozen instance, prototype and class. Only a verified transaction changes state.
E2	**Low**	`applyAll` skipped holes: a "3-transaction" sparse batch applied 1.	Copy the list once, refuse holes.
E3	**Low**	A `Map` as genesis gave a ledger with 0 coins; `null` crashed.	Genesis must be a plain object.
E4	**Low**	`verify` accepted `[360, 361]` as the bytes of "hi"; lone surrogates let one signature fit two strings; a number made "never throws" verify throw.	Only bytes or well-formed text; verify returns `false`.

Checked, not vulnerable: a compressed key with x ≥ p (an alias of a real point) is refused, so one key cannot have two addresses. **Still open (accepted):** random *k* instead of RFC 6979 (low); a wallet signs whatever bytes it is handed (info; real wallets show the user what they sign, EIP-712).

Common mistakes

- **"The signature is valid, so the payment is fine."** Without a nonce and chain id, a valid signature can be replayed forever.
- **Using the signature in the transaction id.** Malleable signatures then change the id (the Mt. Gox 2014 excuse).
- **Floating-point money.** 0.1 + 0.2 ≠ 0.3. Use whole units (satoshi, wei).
- **Checking one object, using another.** Read untrusted input once, freeze it, then verify and use that copy.
- **Public mutable state.** If code can write a balance directly, signatures protect nothing.
- **Reusing the random k.** Two signatures with the same k reveal the private key (Sony PS3, 2010).

5 interview questions

1. Why can't someone derive my private key from my public key?

Because that means solving the elliptic-curve discrete logarithm problem for a 256-bit group. The best known algorithms need about 2^{128} operations, far beyond any computer.

2. What is signature malleability and how do Bitcoin and Ethereum stop it?

If (r, s) is valid, so is (r, n − s). A third party can create the twin without the key. Both chains require the low-S form (BIP 146, EIP-2), and modern ids (SegWit txid) do not cover signatures.

3. What is a replay attack, and what are the two kinds you must block?

Re-submitting a valid signed transaction. Same chain: blocked by a per-account nonce (account model) or by spending each output once (UTXO). Another chain: blocked by signing the chain id (EIP-155).

4. Account model vs UTXO model?

Ethereum stores a balance and nonce per address; Bitcoin stores unspent outputs, each locked to a key, and a payment consumes outputs and creates new ones. UTXO is easier to verify in parallel; accounts are simpler for smart contracts.

5. What does an address checksum protect against, and what not?

It catches typos when a human copies an address (here 9,999 of 10,000 one-character typos). It does not protect against a correct-looking address swapped by malware (address poisoning).

Exercises (solutions tested in `exercises/solutions`)

Level	Exercise	Goal and hint
Easy	Sign a note, not a payment	`signNote` / `verifyNote` for "I, Sita, agree to meet Ram at 5 pm." Prove the note signature can never be used as a payment. *Hint: a different domain string.*
Medium	What goes into the next block?	`selectForBlock(ledger, pending)` for payments arriving out of order, duplicated, or with gaps. *Hint: only the next nonce per sender; repeat passes, because one payment can fund another.* Result: 3 selected, 1 dropped, 1 waiting.
Challenge	A 2-of-3 shared vault	Coins move only when 2 *different* owners sign the exact payment. Refuse a single owner, the same owner twice, outsiders, mismatched amounts and replays. *Hint: the address commits to the threshold and sorted keys.*

Cheat sheet

secp256k1	`y² = x³ + 7 (mod p)` ; group order n ≈ 1.16 × 10^{77}
Private → public	`P = k × G` (fast); P → k is the discrete log problem (infeasible)
Compressed key	33 bytes: `02` / `03` (y even/odd) + 32-byte x
Address	BTC: RIPEMD-160(SHA-256(P)) · ETH: last 20 B of Keccak-256(P) · here: first 20 B of SHA-256(P)
ECDSA signature	(r, s), 64 bytes; valid only for this key and these exact bytes
Low-S	s ≤ n/2 → exactly one valid form per signature
Replay protection	nonce (same chain) + chainId (other chains) + domain (other message types)
Money	whole units only (satoshi, wei); never floats
Checksum address	case of each letter = hash nibble ≥ 8 (EIP-55 idea)
Run	`npm start · npm run attack · npm run typo · npm test`

References

1. SEC 2: Recommended Elliptic Curve Domain Parameters (secp256k1, §2.4.1) — https://www.secg.org/sec2-v2.pdf
2. S. Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System (§2 Transactions) — https://bitcoin.org/bitcoin.pdf
3. A. Antonopoulos, D. Harding, Mastering Bitcoin, 3rd ed., ch. 4 "Keys and Addresses" & ch. 8 "Digital Signatures" — https://github.com/bitcoinbook/bitcoinbook
4. A. Antonopoulos, G. Wood, Mastering Ethereum, ch. 4 "Cryptography" — https://github.com/ethereumbook/ethereumbook
5. ethereum.org, Ethereum accounts (keys, addresses, nonces) — https://ethereum.org/en/developers/docs/accounts/
6. EIP-155: Simple replay attack protection — https://eips.ethereum.org/EIPS/eip-155
7. EIP-55: Mixed-case checksum address encoding — https://eips.ethereum.org/EIPS/eip-55
8. BIP 146: Dealing with signature encoding malleability (low-S) — https://github.com/bitcoin/bips/blob/master/bip-0146.mediawiki
9. RFC 6979: Deterministic usage of DSA and ECDSA — https://www.rfc-editor.org/rfc/rfc6979
10. SWC-121: Missing protection against signature replay attacks — https://swcregistry.io/docs/SWC-121

DAY 04 / 15 · PHASE 1 — FOUNDATIONS

Merkle trees, forks & consensus

Sunday 18 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-04

Learning objectives

1. Build a Merkle root for a list of transactions and explain why one 32-byte hash can stand for all of them.
2. Create and verify a Merkle proof, and explain why its size grows with $\log_2(n)$.
3. Know three classic Merkle-tree attacks (duplicate last node, second preimage, position lies) and the fixes.
4. Explain why forks happen, why the rule is "most total *work*", not "most blocks", and what a reorg undoes.
5. Check a payment like a phone wallet (SPV): headers only + a Merkle proof + confirmations.

The concept, step by step

Step 1 — A knockout tournament for fingerprints

Hash every transaction (a **leaf**). Pair the hashes and hash each pair. Pair the results and hash again until one hash is left: the **Merkle root**. It goes into the block header, so the block hash covers every transaction. Change one transaction and the root changes.

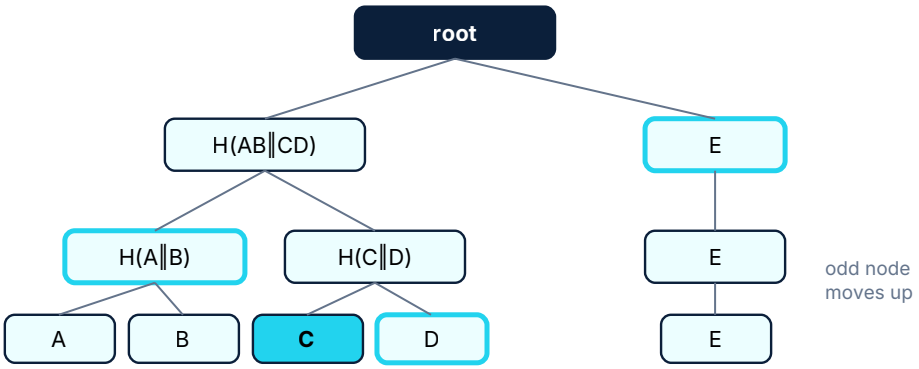


Figure 1 — 5 transactions. To prove C you need only D, H(A‖B) and E (outlined): 3 hashes, not 5 transactions.

Step 2 — A proof is your path to the final

To prove "C is in the block" you give the **siblings** C met on the way up. The verifier rebuilds the path and compares the result with the root in the header. For 1,000 transactions that is 10 hashes (320 bytes); for 1,000,000 it is 20 hashes (640 bytes). Every time the block doubles, the proof grows by one hash.

Step 3 — Tag your hashes

A naive tree can be fooled. Our rules: leaves are `H(0x00‖data)` and inner nodes `H(0x01‖left‖right)` (RFC 6962), so an inner node can never pose as a transaction; an odd node moves up unchanged instead of pairing with itself (Bitcoin's CVE-2012-2459); the root also hashes the **number** of leaves, so a proof cannot lie about the position. Tonight added a fourth rule: the leaf includes the **signature**, not only the transaction id (finding E1).

Step 4 — Forks and the most-work rule

Messages take time to cross the network. If two miners find a block at almost the same moment, half the network sees one and half sees the other: a **fork**. Each node keeps the block it saw first. When the next block lands on one branch, that branch has more total work ($\Sigma\ 2^{bits}$), and every node switches to it: a **reorg**. Payments in the losing block are undone.

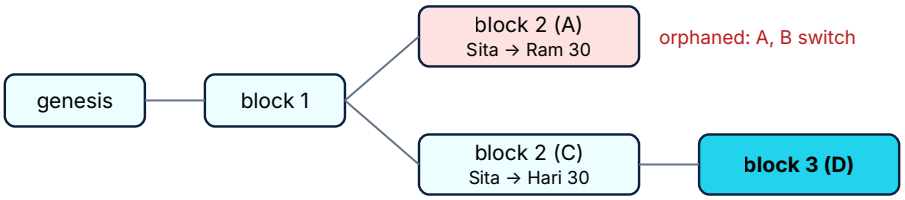


Figure 2 — the double spend from `npm run fork`: Ram's payment was "confirmed" for about 1 second, then disappeared.

Step 5 — A phone wallet (SPV)

A light client keeps only block **headers**, checks their Proof of Work, follows the header chain with the most work, and asks a full node for a Merkle proof of its payment. It learns "paid, N confirmations deep" without downloading anyone else's payments (Bitcoin whitepaper §8).

Key code walkthrough

Verifying a proof (`src/merkle.js`)

```
let h = leafHash(leaf); let i = index; let width = leafCount; let used = 0;
while (width > 1) {
  if (i % 2 === 1)        h = nodeHash(sibs[used++], h);  // we are the RIGHT child
  else if (i + 1 < width) h = nodeHash(h, sibs[used++]);  // we are the LEFT child
  /* else: last node of an odd level, moves up unchanged */
  i >>= 1; width = Math.ceil(width / 2);
}
if (used !== sibs.length) return false;                  // extra hashes = some other tree
return finalRoot(leafCount, h).toString("hex") === rootHex; // root commits to the count
```

Left/right is worked out from `index` and `leafCount` only; the proof carries nothing but hashes, so it cannot "choose" a side.

The evening fix: the leaf covers the signature (`src/block.js`)

```
function txLeaf(tx) {
  if (typeof tx.signature !== "string") throw new TypeError("signature must be a string");
  // morning: Buffer.from(txId(tx), "hex")  ← txId leaves the signature OUT
  return Buffer.from(JSON.stringify([LEAF_DOMAIN, txId(tx), tx.signature]), "utf8");
}
const txRoot = (txs) => merkleRoot(txs.map(txLeaf));
```

Fork choice and remembering invalid blocks (`src/node.js`)

```
// #connect(): fork choice by total work (workOf(bits) = 2^bits)
const work = parent.work + workOf(block.header.bits);
if (work > this.#blocks.get(this.#tip).work) this.#tip = block.hash; // tie: keep first

// receive(): a known-bad block, or a child of one, is refused at once (E2)
if (this.#invalid.has(block.hash)) return { status: "rejected", reason: "known invalid
block" };
if (this.#invalid.has(block.header.prevHash)) this.#markInvalid(block.hash);
```

Light client check (`src/spv.js`)

```
verifyPayment(tx, proof, blockHash) {
  const v = verifyTransfer(tx);                        // a real signed payment (Day 03)
  if (!this.#onBestChain(blockHash)) return { ok: false, reason: "… stale branch" };
  if (!verifyProof(txLeaf(v.tx), proof, entry.header.merkleRoot)) return { ok: false, … };
  return { ok: true, confirmations: this.tip.height - entry.height + 1 };
}
```

Results from the run

Run	Result
`npm start` — 1,000 signed payments	One root `ba173f79…`; proof for #737 = 10 hashes = 320 bytes (100× less than 32,000 bytes of ids); wrong amount → false; wrong position #738 → false
Proof size	1 → 0 hashes · 10 → 4 · 1,000 → 10 · 1,000,000 → 20 (640 bytes)
`npm run fork` — 4 nodes, double spend	Split 2–2 at t = 1.1 s; D mines block 3; A and B each throw away 1 block at t = 2.30 s; all agree; Ram 0, Hari 30
`npm run sim` — 5 miners, 300 blocks, 1 s blocks	Wasted blocks: 10 ms delay 0.3 % · 100 ms 6.0 % · 300 ms 16.0 % · 1,000 ms 37.7 % (deepest reorg 4)
`npm run spv` — 30 blocks, 600 payments	Full blocks 205,611 bytes vs headers 5,986 bytes (34× less); Ram's payment: 5-hash proof, 26 confirmations; 250-coin lie, stale-branch payment and no-work header all refused
Quality	44 / 44 tests pass · ESLint clean · `npm audit` 0 vulnerabilities

QA & security findings

Morning: 8 findings fixed (1 high, 5 medium, 2 low). Evening audit as an outsider: 3 more (1 medium, 2 low), each with a failing test first.

ID	Sev.	Problem	Fix
A3	**High**	"Longest chain" counted blocks and trusted each block's difficulty: 4 blocks worth ~8 guesses beat 2 honest blocks worth ~131,000.	Most total work; difficulty is a network rule.
A1, A2, M1	**Medium**	Duplicate-last trick, inner node passed off as a leaf, proof lying about the position.	Odd node moves up; 0x00/0x01 tags; root commits to the leaf count.
A4, M2	**Medium**	Unlimited orphan pool; a getter-based payment changing after acceptance bricked the node.	PoW-checked, capped pool; frozen copies of every transaction.
E1	**Medium**	Leaves used `txId`, which excludes the signature: a copy with a broken signature had the same block hash and, sent first as an orphan, made the node drop the real block as a "duplicate" (victim stuck at height 1, Ram 0 instead of 30).	Leaf = [txId, signature]: the block hash covers every signature.
E2	**Low**	Invalid blocks were forgotten: 20 resends at height 100 cost 2,044 ms of replays.	Remember permanently invalid blocks and their children (86 ms). Temporary failures are not remembered.
E3	**Low**	`chain()` / `ledgerAt()` crashed on an unknown hash.	Clear `RangeError`.

Known issues kept: ledger replay from genesis per block (K1), orphan / invalid-set eviction by paid PoW (K2), no mempool or deep-reorg limit (K3), 2-hour future timestamps (K4), no byte limit per block (K5), and light clients trust that full nodes validated payments (K6).

Common mistakes

- Pairing the last odd node with itself — two different lists get the same root.
- Hashing leaves and inner nodes the same way — a 64-byte node becomes a fake "transaction".
- Letting the proof say left/right — work it out from index and leaf count.
- Committing to data that leaves something out (here: the signature). Ask "what exactly does this hash cover?"
- Following the chain with the most blocks instead of the most work.
- Treating 1 confirmation as final. Forks are normal; finality is a probability.

Interview questions

1. Why is a Merkle proof O(log n)?

Each level halves the number of nodes, and the proof needs one sibling per level: $\lceil \log_2 n \rceil$ hashes. 1,000 leaves → 10 hashes.

2. What is CVE-2012-2459?

Bitcoin pairs an odd last node with itself, so [a,b,c] and [a,b,c,c] share a root. A node could be fed an invalid "mutated" version of a valid block with the same hash. Fix: move the odd node up (or reject duplicates) and never cache "invalid" for a hash that has a valid twin.

3. Why do Merkle trees use different prefixes for leaves and nodes?

To prevent second-preimage attacks: without them, the 64 bytes of an inner node can be presented as a leaf and verify against the same root (RFC 6962 §2.1).

4. "Longest chain" — longest by what?

By total work (sum of expected hashes, 2^{bits} per block). Counting blocks would let an attacker win with many cheap blocks.

5. What does an SPV client trust?

That the chain with the most work contains only valid transactions. It checks PoW and inclusion, not validity, so it is safe only while honest miners hold most of the hash power.

Exercises summary

Level	Task	Result of the solution
Easy	`proofLength(n, index)` by arithmetic only	Matches `merkleProof` on all 2,080 (size, index) pairs; in a 5-leaf tree leaf 4 needs only 1 hash
Medium	`findDifferences(a, b)`, top-down	1 changed payment of 1,024 found with 21 comparisons; 4 changed with 69
Challenge	Multiproof for many leaves	8 of 1,000 payments: 27 hashes instead of 78; tampering → false

Cheat sheet

leaf	`H(0x00 ‖ JSON[tx-leaf, txId, signature])`
node	`H(0x01 ‖ left ‖ right)`; odd node moves up unchanged
root	`H(0x02 ‖ count(8 bytes) ‖ top)` — in the block header
proof size	$\lceil \log_2 n \rceil \times 32$ bytes (1,000 → 320 B; 1,000,000 → 640 B)
work	2^{bits} per block; best chain = most total work; tie → first seen
reorg	switch branch; blocks above the fork point are undone
SPV	headers + Merkle proof + confirmations = tip height − block height + 1
commands	`npm start · fork · sim · spv · attack:evening · ex:easy|medium|challenge · test`

Key terms

Term	Plain meaning
Merkle root	One hash in the block header that stands for every transaction in the block
Merkle proof	The sibling hashes on the path from one leaf to the root (about $\log_2 n$ of them)
Multiproof	One proof for several leaves; shared hashes are sent once (exercise 3: 27 instead of 78)
Fork	Two valid blocks with the same parent, usually because news travels at a finite speed
Reorg	A node switches to a branch with more work; blocks on the old branch are undone
Orphan (this code)	A block whose parent the node has not seen yet; kept (max 100) and connected later
Stale block	A valid block that ended up off the winning chain: wasted work
Work	Expected guesses to find a block, 2^{bits}; a chain's work is the sum
SPV / light client	Checks payments with headers + Merkle proofs, without every block
Confirmations	How many blocks, counting its own, sit on top of a payment's block

What I'd explore next

- Bitcoin's real 80-byte header, double SHA-256 and the compact `nBits` target; verify a real block's Merkle root.
- Merkle Patricia tries and Verkle trees: Ethereum commits to every balance, so a light client can prove "Sita has 70 coins".
- Selfish mining (Eyal & Sirer, 2014): simulate a miner that hides blocks with today's network code.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", §7–8 — https://bitcoin.org/bitcoin.pdf
2. R. Merkle, "A Digital Signature Based on a Conventional Encryption Function", CRYPTO '87 — https://link.springer.com/chapter/10.1007/3-540-48184-2_32
3. RFC 6962, Certificate Transparency, §2.1 Merkle Hash Trees — https://www.rfc-editor.org/rfc/rfc6962#section-2.1
4. CVE-2012-2459 (duplicate transactions in the Merkle tree) — https://nvd.nist.gov/vuln/detail/CVE-2012-2459
5. BIP 141, Segregated Witness: witness commitment — https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki
6. Bitcoin Developer Guide: Block Chain — https://developer.bitcoin.org/devguide/block_chain.html
7. A. Antonopoulos & D. Harding, Mastering Bitcoin, 3rd ed., ch. 11–12 — https://github.com/bitcoinbook/bitcoinbook
8. ethereum.org: Merkle Patricia Trie — https://ethereum.org/en/developers/docs/data-structures-and-encoding/patricia-merkle-trie/
9. I. Eyal & E. G. Sirer, "Majority is not Enough: Bitcoin Mining is Vulnerable" (2014) — https://arxiv.org/abs/1311.0243

DAY 05 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

Ethereum & the EVM

Monday 19 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-05

Learning objectives

1. Explain the two kinds of Ethereum account (EOA and contract) and what a nonce does.
2. Read EVM bytecode as a list of opcodes and trace what each one does to the stack.
3. Work out the gas of a transaction: the 21,000 base cost, cheap opcodes, expensive storage, cold vs warm access and refunds.
4. Find any Solidity variable in raw storage: packed slots, mappings at `keccak256(key, slot)`, arrays at `keccak256(slot) + i`.
5. Prove a model is right by testing it against the real thing (differential testing).

The concept, step by step

Step 1 — From a coin ledger to a world computer

Days 01–04 built a ledger that only tracks **who owns how many coins**. Ethereum keeps that ledger and also **runs programs** on it. Every node runs the same small computer, the **EVM** (Ethereum Virtual Machine), on the same inputs, and every node must get exactly the same result. If one node got a different answer, the network would split.

Step 2 — Two kinds of account

	Externally owned account (EOA)	Contract account
Controlled by	A private key (a person or a bot, Day 03)	Its own code
Has code?	No	Yes, fixed once deployed
Has storage?	No	Yes, 2^{256} slots of 32 bytes
Balance + nonce	Yes: nonce = transactions sent	Yes: nonce = contracts it created
Starts a transaction?	Yes	No, only reacts to calls

The **nonce** is a counter. Each transaction must use the next number, so the same signed transaction can never be played twice. A contract's address is also worked out from its creator and nonce: `keccak256(rlp(sender, nonce))`. In my run the predicted and real address matched exactly.

Step 3 — The EVM is a calculator with a stack

Contract code is a list of one-byte instructions called **opcodes** (`0x01` = `ADD`, `0x55` = `SSTORE`). Each opcode takes numbers off the top of a stack of 256-bit numbers and puts its answer back on top. Here is "store 2 + 3 in slot 0":



Figure 1 — The stack after each opcode. Top of the stack is highlighted. Total execution gas: 3+3+3+3+22,100.

Step 4 — Gas: the price list that keeps the computer alive

Every opcode costs **gas**. You pay `gas used × gas price`. Gas does two jobs: endless loops stop because they run out of gas, and spam costs real money because thousands of nodes repeat your work. A plain ETH transfer always costs exactly **21,000** gas.

The biggest costs are in storage, because every node must keep that data for ever. The first time a transaction touches a slot it is **cold** (2,100 extra); after that it is **warm** (100), from EIP-2929.

SSTORE write (cold slot)	Gas	Why
0 → x (new value)	22,100	A new 32-byte value every node must keep
x → y (change)	5,000	The slot already exists
x → x (no change)	2,200	You only pay for the access
x → 0 (clear)	5,000 − 4,800 refund	Freeing state is rewarded (EIP-3529: refund ≤ ⅕ of gas used)

Step 5 — Storage: a wall of lockers

A contract has 2^{256} lockers (slots) of 32 bytes, all zero until written. Solidity fills them in the order variables are declared and **packs** small values into one slot when they fit. Mappings and dynamic arrays cannot be stored in order (they can grow without limit), so their items are placed at `keccak256` addresses where they will never collide.

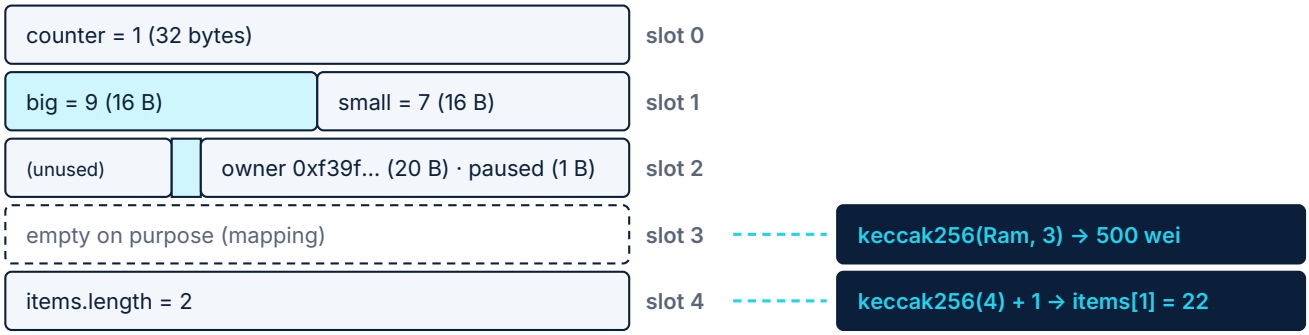


Figure 2 — Real storage of Layout.sol read with eth_getStorageAt. Packed values share a slot; mappings and arrays are scattered by keccak256.

Key code walkthrough

1. The gas price list (src/evm.js)

```
// Mainnet gas constants. Storage is priced 1,000× higher than arithmetic.
zero: 0, base: 2, verylow: 3, low: 5, mid: 8, high: 10, jumpdest: 1,
coldSload: 2100, warmRead: 100, sset: 20000, sreset: 2900, clearRefund: 4800,
const MAX_GAS = 30_000_000;  // one block; also caps memory at ~4 MB (QA fix M1)
```

2. Cold vs warm inside SSTORE

```
// First touch of a slot in this transaction adds the cold surcharge, then it is warm.
if (!s.warm.has(key)) { s.warm.add(key); cost += G.coldSload; }
if (value === 0n) s.refund += G.clearRefund;     // clearing a slot earns a refund
// at the end: refund is capped at 1/5 of the gas used (EIP-3529)
const refund = Math.min(Math.max(s.refund, 0), Math.floor(raw / G.maxRefundQuotient));
```

3. Finding a variable in storage (src/storage.js)

```
// balances[key] when the mapping is declared at `slot`
function mappingSlot(keyType, key, slot) {
  return BigInt(keccak256(coder.encode([keyType, "uint256"], [key, slot])));
}
// items[i] when the array is declared at `slot`: keccak256(slot) + i
function arraySlot(slot, index) {
  const start = BigInt(keccak256(zeroPadValue(toBeHex(slot), 32)));
  return (start + BigInt(index)) % (1n << 256n);
}
// read a packed field: shift right by its offset, keep `size` bytes
function unpack(word, offset, size) {
  return (getBigInt(word) >> BigInt(8 * offset)) & ((1n << BigInt(8 * size)) - 1n);
}
```

4. The contract that shows the layout (contracts/Layout.sol)

```
uint256 public counter;               // slot 0
uint128 public small; uint128 public big;  // slot 1, packed: small low 16 B, big high 16 B
address public owner; bool public paused;  // slot 2, packed: 20 + 1 bytes
mapping(address => uint256) public balances; // slot 3 stays empty
uint256[] public items;              // slot 4 = length
```

Results from my run

What I measured	Result
Sita sends Ram 1 ETH	21,000 gas × 1.875 gwei = fee 0.000039375 ETH; nonce now 1
Predicted vs real contract address	0xe7f1725E…0512 — **match**
Deploying Layout.sol	736,081 gas, 2,952 bytes of code
`increment()` 0 → 1, then 1 → 2	43,567 gas, then 26,467 gas — **39% cheaper** the second time
"2 + 3, store, read, return" on my mini-EVM vs Hardhat	43,230 vs 43,230 gas — **match**, returned 5
Differential fuzz test	150 random programs, 10,134 opcode steps in 300 transactions — every step's gas identical

Quality	45 / 45 tests pass, lint clean, npm audit 0, Slither 0 high/medium/low

Two surprises: the same function costs much less the second time (the first call *creates* a value, the second only *changes* it), and `SLOAD` right after `SSTORE` costs just 100, because the slot is already warm.

QA & security findings

ID	Severity	Found by	Problem in plain words	Fix
M1	**Medium**	Manual	No upper gas limit: one MSTORE could make Node try to allocate 4 GB of memory.	gasLimit must be 21,000 – 30,000,000 (one block).
D1	**Low**	Differential test	Running off the end of code: the real EVM records an implicit STOP, mine did not.	Record the implicit STOP.
L1	**Low**	Manual	A long loop could keep millions of trace steps in memory.	Keep at most 100,000 steps, flag `traceTruncated`.
L2	**Low**	Manual	Storage with Number keys (0 instead of 0n) silently read as zero.	Require `Map<bigint, bigint>`.
L3	**Low**	Manual	Code starting with 0xEF was accepted though every chain refuses it (EIP-3541).	Refuse it up front with a clear error.

Also attacked and found safe: re-entering `withdraw`, owner-only functions, overflow, replay across chains (EIP-155) and with a used nonce, jumps into PUSH data, and ETH forced in by `selfdestruct`.

Common mistakes

- **Thinking gas price = gas.** Gas measures work; the price per unit changes with demand. Fee = gas used × gas price.
- **Forgetting the first write is expensive.** Setting a slot from 0 costs 22,100; designs that create many new slots get costly fast.
- **Reordering variables in an upgradeable contract.** The slot numbers move and old data is read as the wrong thing (Day 11).
- **Trusting a model you never compared with reality.** My unit tests passed; only the differential test caught D1.
- **Using `transfer` to send ETH.** Its 2,300-gas limit breaks smart-contract wallets; use `call` and check the result.

Interview questions

1. Why does the EVM need gas at all?
Without it a single loop could freeze every node for ever. Gas makes every step cost something, so loops end and people pay for the work they ask the network to do.

2. Where does Solidity store `balances[alice]` if the mapping is declared at slot 3?
At `keccak256(abi.encode(alice, 3))`. Slot 3 itself stays empty.

3. Why does the second `increment()` cost less than the first?
The first changes the slot from zero (a new value, 20,000 + cold 2,100); the second only changes an existing value (2,900 + cold 2,100).

4. What is the difference between an EOA and a contract account?
An EOA is controlled by a private key and can start transactions; a contract account is controlled by its code, has storage, and only runs when called.

5. How can you be sure a home-made EVM is correct?
Differential testing: run the same random programs on it and on a reference EVM, and require identical gas at every step, the same storage and the same output.

Exercises

Level	Goal	Hint
Easy	Read `small` and `big` from packed slot 1. Expected: 7 and 9.	`unpack(word, 0, 16)` and `unpack(word, 16, 16)`; the first variable sits at the low end.
Medium	Write to slot 0, read it twice, return it; then clear it.	Look at the trace: first access 2,100 extra, then 100. Check the refund cap of ⅕.
Challenge	Add `MUL` or `AND` to the mini-EVM.	Unit-test stack, gas and 256-bit wrapping, then add a differential test against Hardhat.

Cheat sheet

Costs to remember
Plain transfer: 21,000
ADD / PUSH: 3 · MUL: 5
Cold storage access: +2,100 · warm: 100
New value 0 → x: 22,100 (cold)
Clear refund: 4,800, max ⅕ of gas used
Block limit: 30,000,000

Where data lives
Variables: slot 0, 1, 2 … in order
Small types: packed into one slot
`mapping` at slot p: `keccak256(key, p)`
`array` at slot p: length at p, item i at `keccak256(p) + i`
Contract address: `keccak256(rlp(sender, nonce))`

References

1. Ethereum.org — Ethereum Virtual Machine: https://ethereum.org/en/developers/docs/evm/
2. Ethereum.org — Gas and fees: https://ethereum.org/en/developers/docs/gas/
3. Ethereum Yellow Paper (Appendix G fee schedule, Appendix H instruction set): https://ethereum.github.io/yellowpaper/paper.pdf
4. evm.codes — interactive opcode reference with gas costs: https://www.evm.codes/
5. Solidity docs — Layout of state variables in storage: https://docs.soliditylang.org/en/v0.8.24/internals/layout_in_storage.html
6. EIP-2929 — Gas cost increases for state access opcodes: https://eips.ethereum.org/EIPS/eip-2929
7. EIP-3529 — Reduction in refunds: https://eips.ethereum.org/EIPS/eip-3529
8. EIP-2200 — Structured definitions for net gas metering: https://eips.ethereum.org/EIPS/eip-2200
9. ethers.js v6 documentation: https://docs.ethers.org/v6/
10. Andreas M. Antonopoulos & Gavin Wood, *Mastering Ethereum*, ch. 13 "The EVM": https://github.com/ethereumbook/ethereumbook

DAY 06 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

Solidity fundamentals

Tuesday 20 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-06

Learning objectives

1. Explain state vs local variables and where data lives: `storage`, `memory`, `calldata`.
2. Use mappings, structs, enums, arrays, constants and immutables, and know what each costs.
3. Control access with visibility, `view`/`pure`, `payable` and modifiers.
4. Fail loudly with custom errors; emit events in an order that apps can replay.
5. Send ETH safely with checks → effects → interaction plus a reentrancy guard, and prove it with an attacker contract.
6. Test like an attacker: boundaries, control experiments, random tests against a model, and mutation testing.

The concept, step by step

Step 1 — A contract is a program with its own bank account and memory

Its **state variables** sit in storage, the 32-byte lockers from Day 05, and survive between calls. **Local variables** exist only while a function runs. A `constructor` runs once at deployment. An `immutable` (like `owner`) is set there and copied into the bytecode, so it uses no storage slot.

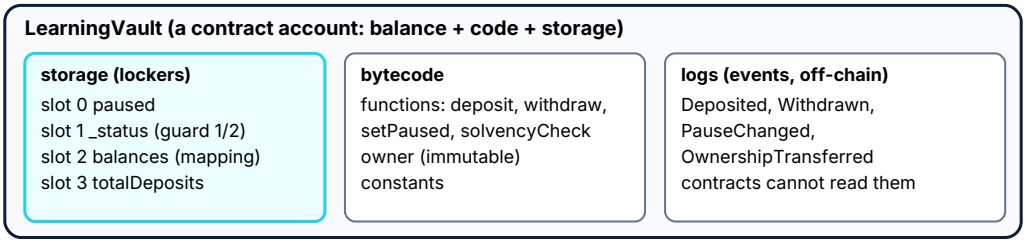


Figure 1 — Where each part of LearningVault lives. Storage is read raw with `eth_getStorageAt` in the tests.

Step 2 — Types, and where data lives

Value types (`uint256`, `bool`, `address`, `enum`) are copied. **Reference types** (arrays, structs, `string`) need a location: `storage` is the permanent on-chain data (a `storage` pointer edits the real thing), `memory` a temporary copy, `calldata` read-only call input and the cheapest for external functions. A `mapping` is a lookup table you cannot list. `StudyLog.sol` shows all of these in one small contract. Names are measured in **bytes**: 22 characters of "€" are 66 bytes and fail a 64-byte limit.

Step 3 — Who may call what

`external`/`public` decide who can call; `view` promises not to change state, `pure` not even to read it; `payable` lets ETH in. **Modifiers** wrap a function in reusable checks: `onlyOwner`, `whenNotPaused`, `nonReentrant`. They save repetition but do not replace testing every caller.

Step 4 — Fail loudly, and keep an honest diary

A `revert` undoes everything the call did. **Custom errors** such as `InsufficientBalance(available, requested)` carry data and cost less than text. **Events** are the public diary apps read; their *order* matters because apps replay them.

Step 5 — Sending ETH hands control to someone else's code

The receiver's `receive()` runs in the middle of your function. If your books still say "balance 1" it can ask again. Order: **check → update your state → send**, plus a guard as a second lock.

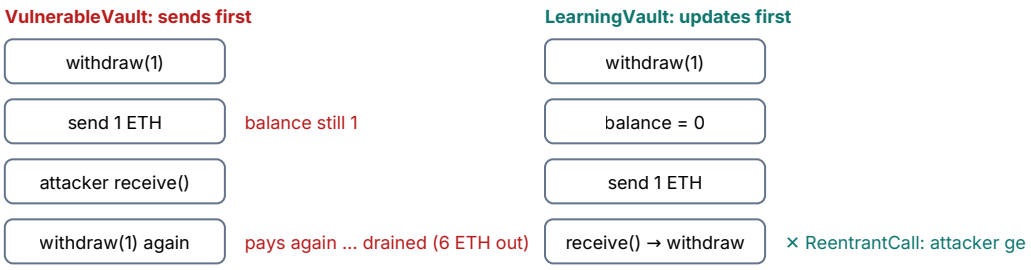


Figure 2 — The same attacker contract against both vaults (measured in `npm start`).

Key code walkthrough

The safe withdrawal (`contracts/LearningVault.sol`)

```
function withdraw(uint256 amount) external override nonReentrant {
    if (amount == 0) revert ZeroAmount();                                  // 1 checks
    uint256 available = balances[msg.sender];
    if (amount > available) revert InsufficientBalance(available, amount);

    uint256 remaining = available - amount;                                // 2 effects
    balances[msg.sender] = remaining;
    totalDeposits -= amount;
    emit Withdrawn(msg.sender, amount, remaining);                         //   event BEFORE the
call

    (bool success, ) = payable(msg.sender).call{value: amount}("");        // 3 interaction
    if (!success) revert TransferFailed();
}
```

The guard flips 1 ↔ 2, never to 0

```
modifier nonReentrant() {
    if (_status == _ENTERED) revert ReentrantCall();   // _ENTERED = 2
    _status = _ENTERED;  _;  _status = _NOT_ENTERED;    // _NOT_ENTERED = 1: slot never empty
}
```

A slot that goes 0 → 1 → 0 pays a fresh write each call (Day 05): 52,031 gas per withdrawal vs 42,859 with 1 ↔ 2.

Results from the run

Run	Result
`npm start` deploy	376,377 gas; 1,385 bytes of runtime code; `owner` is in the bytecode, not in a slot
Deposits	first 69,667 gas (0 → value), second 35,467 gas (value → value)
Reentrancy	VulnerableVault: attacker ends with 6.0 ETH after 5 re-entries, vault holds 0. LearningVault: `ReentrantCall`, attacker gets its own 1 ETH, vault keeps 5
Failed payout	Recipient with no `receive()` → `TransferFailed`, recorded balance unchanged
Pause	Deposits → `ContractPaused`; withdrawals still work; non-owner → `NotOwner`
Quality	75 / 75 tests · ESLint + Solhint clean · `npm audit --omit=dev` 0 · Slither: no real issue · mutation testing 16 / 16 caught

How I tested it

1. A control before a claim

"The attack fails" means nothing if the attacker could not win anywhere. The tests first drain a deliberately broken `VulnerableVault` (6 ETH out), then show `LearningVault` refusing the same attacker with `ReentrantCall`. The same idea checks the enum: a hand-built call with value 2 works, value 3 is refused.

2. Random operations against a model

200 random programs of deposits, withdrawals and pause toggles run on the contract and on a 15-line JavaScript model. After every step both must agree on balances, `totalDeposits`, the contract's real ETH and on which custom error a failing call returns. (fast-check, fixed seed, so a failure is reproducible.)

3. Break it on purpose

`npm run qa:mutation` changes the contract one line at a time (remove the guard, send before updating, `>` to `>=`, drop an event …) and runs every test. 16 broken versions, 16 caught. The first 19 tests caught only 10 of 15 equivalent breaks: green did not mean watching.

```
$ npm run qa:mutation
KILLED  (1 test failed)   reentrancy guard removed
KILLED  (2 tests failed)  interaction before effects (guard kept)
KILLED  (1 test failed)   failed payout is ignored
...
16 of 16 mutants killed.
```

QA & security findings

The morning's 19 green tests let **5 of 15** broken versions of the contract through. Evening review found 6 issues (full table in `QA.md`).

ID	Sev.	Problem	Fix
M1	**Medium**	Pause blocked `withdraw` too. With an immutable owner, a lost or malicious key locked all ETH forever.	Pause stops deposits only; exits always open.
L1	**Low**	`Withdrawn` emitted after the call: a re-depositing receiver made the log read *Deposited, Deposited(newBalance 50), Withdrawn*, a history that never happened.	Emit with the other effects, before the call.
L2	**Low**	Guard 0 → 1 → 0 paid a new storage write each call: 52,031 gas.	Guard 1 ↔ 2: 42,859 gas (−17.6 %).
L3	**Low**	A constructor test that could never fail; reentrancy guard, failed payout and forced ETH untested.	Real assertions, attacker + control vault, 16 mutants all caught.
I1, I2	**Info**	Dead `ZeroAddress` error and a wrong comment; a hand-rolled lock in `VaultPlus` flagged by Slither.	Removed / modifier used.

Accepted: not independently audited; the owner can stop new deposits; a pause cannot stop an exploit that goes through `withdraw`. Do not deploy with real funds.

Common mistakes

- Sending ETH first and updating the balance after: the classic reentrancy bug.
- Trusting a green test that "the attack fails" without first showing the attack works on a weak target.
- A pause that also freezes withdrawals: an owner-key problem becomes a user-funds problem.
- Emitting an event after an external call; apps that replay logs see events out of order.
- Assuming `address(this).balance` equals your tracked deposits: anyone can force ETH in with `selfdestruct`.
- Looping over a list anyone can grow inside a state-changing function (denial of service).
- Counting characters instead of bytes in a length limit.

Interview questions

1. What is checks-effects-interactions and why does it matter?

Validate, then update your own state, then make external calls. During the call the other side's code runs; if your state is already final it cannot exploit a stale balance.

2. Why use both a guard and the ordering?

Two independent layers: remove either and the other still stops the drain. Mutation tests check each layer separately.

3. `immutable` vs `constant` vs a normal variable?

`constant` is fixed at compile time, `immutable` at deployment; both live in the bytecode, so reading them is cheap. A normal variable occupies a storage slot.

4. Why are custom errors preferred over `require` with text?

Cheaper to deploy and to revert, and they carry typed data that tests and apps can decode.

5. When is pausing withdrawals justified?

To stop a live drain, but it freezes honest users and, if the key is lost while paused, forever. Safer designs add a timelock, a multisig or an auto-expiring pause.

Exercises summary

Level	Task	Result of the solution
Easy	`depositCount`, `getBalance`, immutable vs mutable	3 deposits → count 3; failed deposits are not counted
Medium	Minimum deposit error, indexed-event test, reverted withdrawal changes nothing	`BelowMinimum(999, 1000)`; Deposited log has 2 topics
Challenge	Attacker contract, pause trade-off, two-step ownership	Drains the broken vault (6 ETH), refused by the real one; a typo address can never take over

Cheat sheet

`external / public`	who can call · `view` reads · `pure` reads nothing · `payable` accepts ETH
`revert Name(args)`	undo everything, with typed data · `emit Event(...)` writes a log
mapping slot	`keccak256(abi.encode(key, slot))`
ETH send	`(bool ok,) = to.call{value: v}(""); if (!ok) revert …;` after updating state
commands	`npm start · test · lint · slither · qa:mutation · ex:easy|medium|challenge`

Key terms

Term	Plain meaning
State variable	A value kept in on-chain storage between calls
Modifier	A reusable check wrapped around a function
Custom error	A named revert reason carrying data
Event	A log entry apps can read; `indexed` arguments are searchable
Interface	Function signatures with no code; implementers must provide them
Reentrancy	A paid contract calls back before you finished updating your books
Mutation testing	Break the code on purpose; a good test suite fails every time
Forced ETH	ETH pushed into a contract by `selfdestruct` even if it refuses payments

What I'd explore next

- Day 07: ERC-20 tokens, allowances and OpenZeppelin, with fuzz tests.
- Day 09: reentrancy and access-control exploits in depth, and a timelock or multisig owner.
- Day 11: custom-error and storage-packing gas savings; upgradeable proxies.

References

1. Solidity docs 0.8.24: Structure of a Contract, Contracts, Types. https://docs.soliditylang.org/en/v0.8.24/
2. Solidity docs: Security considerations (reentrancy). https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
3. SWC-107: Reentrancy. https://swcregistry.io/docs/SWC-107
4. OpenZeppelin ReentrancyGuard (why 1 and 2). https://docs.openzeppelin.com/contracts/5.x/api/utils#ReentrancyGuard
5. Hardhat documentation. https://hardhat.org/docs · ethers.js v6. https://docs.ethers.org/v6/
6. Slither static analyzer. https://github.com/crytic/slither
7. EIP-2929 / EIP-3529 (storage gas prices, from Day 05). https://eips.ethereum.org/EIPS/eip-2929

DAY 07 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

ERC-20 tokens

Wednesday 21 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-07

Learning objectives

1. Explain what a token really is (a balance table in a contract) and what ERC-20 promises.
2. Use decimals correctly: the chain stores whole numbers, wallets add the dot.
3. Understand allowances: `approve`, `transferFrom`, the unlimited approval, and the approve race.
4. Use `permit` (EIP-2612) to approve with a signature, and survive its front-running.
5. Build a vault for any token that copes with fee-on-transfer, no-return, false-return and callback tokens.
6. Test a standard, not a contract: one behaviour suite, differential testing and model-based random tests.

The concept, step by step

Step 1 — A token is a table inside a contract

There are no coins to hold. "Ram owns 5 tokens" only means `balanceOf[Ram] == 5 × 10¹⁸`. ERC-20 is a promise about function names and two events, so every wallet and exchange can talk to every token the same way. In `MiniToken` every change goes through one function, `_update(from, to, value)`: `from == 0` is a mint, `to == 0` a burn.

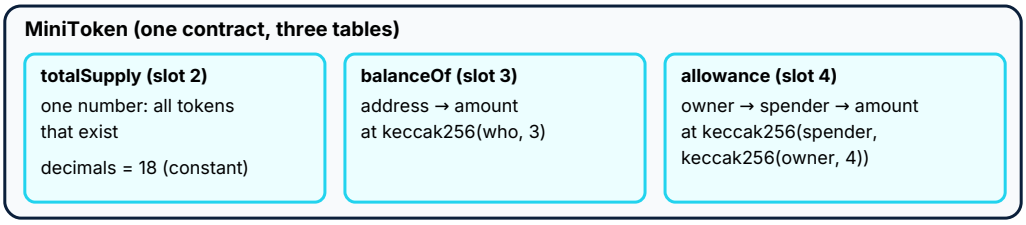


Figure 1 — Where MiniToken keeps its data. The tests open these slots with `eth_getStorageAt` (Day 05).

Step 2 — Decimals are a display trick

The EVM has no fractions. `decimals = 18` tells a wallet to show `balance ÷ 10¹⁸`. 2.5 tokens is 2,500,000,000,000,000,000 units at 18 decimals and 2,500,000 at USDC's 6. Dividing 1 token by 3 gives 333,333,333,333,333,333 each and one unit left over: integer division rounds down.

Step 3 — Letting someone else spend your tokens

`approve(spender, 100)` only records a permission; nothing moves. Later the spender calls `transferFrom(owner, to, 40)`: the token checks the allowance, lowers it to 60, and moves the tokens. The allowance is re-checked at every use against the balance *at that moment*. An allowance of `2²⁵⁶ − 1` means "unlimited" and is never reduced (saves a storage write, but exposes everything you will ever hold).

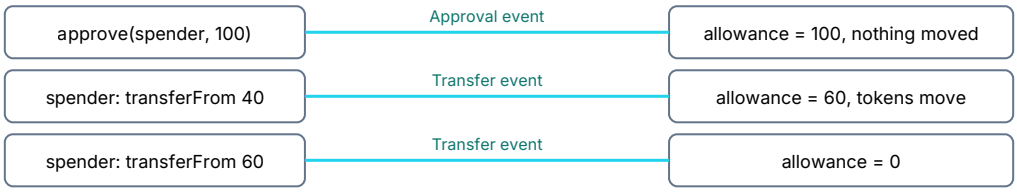


Figure 2 — Spending an allowance: the token lowers it as it goes; a third spend would revert.

Step 4 — The approve race

Alice has approved Mallory for 100 and now sends `approve(50)` to lower it. Mallory sees it pending and front-runs with `transferFrom(100)`; then Alice's `approve(50)` lands and Mallory spends the *new* 50. Total taken: 150, though Alice only ever meant 100 or 50 (measured: 150 units). Fixes: `increase/decreaseAllowance`, a compare-and-set `approveIfCurrent`, or `permit`.

Step 5 — Permit: approve with a signature

EIP-2612: the owner signs a typed message (the Day 03 kind of signature) naming spender, value, a **nonce** and a **deadline**. Anyone can submit it to `permit`; the token recovers the signer, sets the allowance and bumps the nonce, so the signature works once. The owner sends no transaction and pays no gas. The catch: once broadcast, the signature is public, so anyone can use it first.

Key code walkthrough

The one place balances change (`contracts/MiniToken.sol`)

```
function _update(address from, address to, uint256 value) internal virtual {
    if (from == address(0)) totalSupply += value;                    // mint
    else {
        uint256 fromBalance = balanceOf[from];
        if (fromBalance < value) revert ERC20InsufficientBalance(from, fromBalance, value);
        unchecked { balanceOf[from] = fromBalance - value; }         // checked just above
    }
    if (to == address(0)) unchecked { totalSupply -= value; }        // burn
    else unchecked { balanceOf[to] += value; }                       // sum of balances ==
totalSupply
    emit Transfer(from, to, value);
}
```

Credit what ARRIVED (`contracts/TokenVault.sol`)

```
uint256 before = token.balanceOf(address(this));
token.safeTransferFrom(msg.sender, address(this), amount);   // handles no-return and false-
return tokens
received = token.balanceOf(address(this)) - before;          // fee tokens deliver less than
`amount`
if (received == 0) revert NothingReceived();                 // a token can "succeed" and move
nothing
balances[token][msg.sender] += received;
```

Permit that cannot be griefed

```
try IERC20Permit(address(token)).permit(msg.sender, address(this), amount, deadline, v, r, s) {}
catch {}
return _deposit(token, amount);   // if the allowance is really missing, transferFrom reverts
clearly
```

Results from the run

Run	Result
Gas: transfer	To a new holder 51,543; to an existing holder 34,443 (a new slot costs ~17,000+ more). Hand-written token 34,443 vs OpenZeppelin 34,460
Gas: allowance	transferFrom with a finite allowance 40,488; with an unlimited one 37,249
Approve race	Spender ends with 150 units after "100, then lower to 50"
Fee token (1 %)	Naive vault: credits 2000, holds 1980, the last user cannot withdraw. TokenVault: credits 1980, solvent
Weird tokens	USDT-style (no return): TokenVault deposits via SafeERC20. Returns false: `SafeERC20FailedOperation`. Ghost token: `NothingReceived`
Hook token	Re-entry via withdraw, deposit and depositWithPermit refused; inside the callback the vault already shows balance 0, total 0
Quality	167 / 167 tests · ESLint + Solhint clean · `npm audit --omit=dev` 0 · Slither: no real issue · mutation testing 34 / 34 caught

How I tested it

1. One behaviour suite, three tokens

31 checks describe ERC-20 itself (events, boundaries, error data, allowances). The same file runs against `MiniToken`, plain OpenZeppelin and `LearningToken`: if all three pass, the checks describe the standard, not one contract.

2. Differential testing against OpenZeppelin

150 random programs run on MiniToken and OpenZeppelin side by side. After every operation they must agree on success, error name and arguments, events, every balance and allowance, and `totalSupply`. OpenZeppelin is the judge of my token.

3. Random operations against a model

200 random programs on `LearningToken` versus a JavaScript model of a capped ERC-20: sum of balances equals supply, supply never passes the cap, a failed call changes nothing.

4. Controls, then mutants

Every "the vault survives X" has a naive vault that fails X. Then `npm run qa:mutation` breaks the contracts 34 ways. First pass: 31 caught, 3 survived (an untested `NothingReceived` and two untested reentrancy guards). After adding tests: 34 / 34.

QA & security findings

I wrote code and tests in the morning and attacked both in the evening, so the findings are things I designed for but had not proven.

ID	Sev.	Problem	Fix
T1	Low	`NothingReceived` was never exercised: removing the check changed no test result.	`GhostToken` (returns true, moves nothing) plus a naive-vault control.
T2	Low	Reentrancy guards on `deposit` and `depositWithPermit` untested: the attacker only re-entered `withdraw`.	Attacker modes that re-enter each entry point from the token callback.
I1	Info	Depositing "a token" that is an ordinary account was untested (it reverts).	Test added.

Slither: 3 results in production code, all in `_deposit`: the balance-before/after pattern must write state after the external call, protected by `nonReentrant`; the `== 0` check is intended. **Accepted:** the approve race is inherent to ERC-20; rebasing tokens are unsupported; the owner can mint up to the cap; not audited. Do not deploy with real funds.

Common mistakes

- Crediting the `amount` a user asked to deposit instead of what arrived (fee-on-transfer tokens).
- Calling `token.transfer(...)` through a plain `IERC20` on USDT: the empty return value makes the call revert. Use `SafeERC20`.
- Treating `approve` as additive, or lowering an allowance while the spender may be spending it.
- Giving unlimited approvals to contracts you do not fully trust.
- Calling `permit` without a fallback: a front-run signature makes the whole transaction revert.
- Assuming `permit` worked: some tokens have a fallback that "succeeds" without setting anything.
- Forgetting that a token can call you back (ERC-777 style hooks) in the middle of your function.

Interview questions

1. What does `approve` do, and why does it overwrite?

It records how much a spender may still move. It sets, not adds, so the allowance is exactly what you last said; the cost is the race when you change a non-zero value.

2. Why must a vault measure the balance before and after?

Because a token may deliver less than requested (fees) or nothing at all while reporting success. Only the balance difference says what arrived.

3. What is wrong with `token.transfer(...)` on USDT?

USDT returns nothing, but `IERC20` expects a `bool`; decoding empty data reverts. `SafeERC20` accepts "no data" as success and "false" as failure.

4. How can `permit` be front-run, and what is the defence?

The signature is public once broadcast, so someone submits it first and uses up the nonce. Wrap `permit` in `try/catch` and let the later `transferFrom` decide.

5. What is differential testing?

Running two implementations of the same specification on identical random inputs and requiring identical results; any difference is a bug in one of them.

Exercises summary

Level	Task	Result of the solution
Easy	increase/decreaseAllowance, burnFrom, explain decimals	100 → 150 → 120; decrease(999) reverts; 2.5 tokens = 2.5×10^{18} units
Medium	Capped owner-only mint, approve race and `approveIfCurrent`, supply invariant	Mint stops at the cap; compare-and-set reverts after the front-run
Challenge	Naive vault vs fee token, permit front-running, differential test	Naive: credited 2000, holds 1980; TokenVault solvent; deposit survives the front-run

Cheat sheet

`transfer(to, v)`	move your tokens · `Transfer(from, to, v)`
`approve(s, v)`	set allowance (overwrites) · `Approval(owner, s, v)`
`transferFrom(f, t, v)`	needs allowance[f][caller] ≥ v; lowers it (unless $2^{256} - 1$)
mapping slots	`balanceOf` at keccak256(who, 3) · `allowance` at keccak256(spender, keccak256(owner, 4))
commands	`npm start · test · lint · slither · qa:mutation · ex:easy|medium|challenge`

Key terms

Term	Plain meaning
ERC-20	The standard functions and events of a fungible token
Base unit	The smallest indivisible amount: 1 token = 10^{decimals} units
Allowance	How much a spender may still move from an owner
Approve race	Changing an allowance as it is being spent can let a spender take both amounts
Permit	Approve with a signed message; the nonce makes it one-time
Fee-on-transfer	A token whose receiver gets less than the sender sent
SafeERC20	A wrapper that handles tokens returning nothing or false
Differential testing	Two implementations, identical inputs, identical results required

What I'd explore next

- Day 08: ERC-721 and ERC-1155, safe transfers and receiver hooks, royalties.
- Day 09: front-running and access-control exploits in depth; timelocks and multisig owners.
- Later: ERC-4626 vaults (share accounting), and the inflation attack on them.

References

1. EIP-20: Token Standard. https://eips.ethereum.org/EIPS/eip-20
2. EIP-2612: Permit extension for EIP-20. https://eips.ethereum.org/EIPS/eip-2612 · EIP-712. https://eips.ethereum.org/EIPS/eip-712
3. ERC-6093: Standard custom errors for tokens. https://eips.ethereum.org/EIPS/eip-6093
4. OpenZeppelin Contracts 5.x: ERC20 and SafeERC20. https://docs.openzeppelin.com/contracts/5.x/erc20
5. SWC-114: Transaction order dependence. https://swcregistry.io/docs/SWC-114
6. Solidity docs 0.8.24: Security considerations. https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
7. fast-check: property-based testing. https://fast-check.dev/ · Slither. https://github.com/crytic/slither

DAY 08 / 15 · PHASE 2 — ETHEREUM & SOLIDITY

NFTs: ERC-721 and ERC-1155

Thursday 22 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-08

Learning objectives

1. Explain an NFT as an ownership table (`tokenId → owner`) plus a metadata link.
2. Use the two permission types correctly: `approve` (one token) and `setApprovalForAll` (everything).
3. Know why `safeTransferFrom` exists, and why its receiver call is also a re-entrancy door.
4. Build a paid collection: supply cap, price, per-wallet limit, metadata, royalties (ERC-2981).
5. Use ERC-1155 for many item types (coins and one-of-ones) in one contract, with batches.
6. Test a standard, not a contract: one suite and a differential fuzz test against OpenZeppelin.

The concept, step by step

Step 1 — An NFT is a table

Day 07's ERC-20 stored `address → amount`. ERC-721 stores `tokenId → owner`: every row is one numbered item with exactly one owner. "Ram owns NFT #7" simply means `ownerOf(7) == Ram`. A token that was never minted (or was burned) has no owner, and `ownerOf` **reverts** rather than answering "nobody".

ERC-20 (Day 07)

```
address → amount
Ram → 500
Sita → 120
```

ERC-721 (today)

```
tokenId → owner
#1 → Ram
#2 → Ram
#3 → Sita
```

ERC-1155 (today)

```
(id, owner) → amount
(GOLD, Ram) → 1000
(SWORD, Ram) → 5
(CROWN, Ram) → 1
```

Figure 1 — Three token standards, three tables. The shape of the table is the whole difference.

Step 2 — The picture is not on the chain

`tokenURI(7)` returns a **link** to a JSON file (name, description, image link). The chain proves *who owns #7*; the JSON says *what* #7 is. An `ipfs://` link is a content hash, so the file cannot be swapped silently; an `https://` link can. That is why today's evening improvement adds `freezeMetadata()`.

Step 3 — Two kinds of permission

`approve(Hari, 7)` lets Hari move token #7 once; it is wiped automatically when #7 moves. `setApprovalForAll(market, true)` lets an operator move **every** token you own, including ones you receive later. Marketplaces need it, and phishing sites abuse it: one signature can empty a wallet.

Step 4 — Safe transfers, and the door they open

Plain `transferFrom` into a contract that does not understand NFTs locks the token there forever. `safeTransferFrom` first calls `onERC721Received` on the receiver and refuses unless it returns its own 4-byte selector. But that call hands control to the receiver *in the middle of your function*:

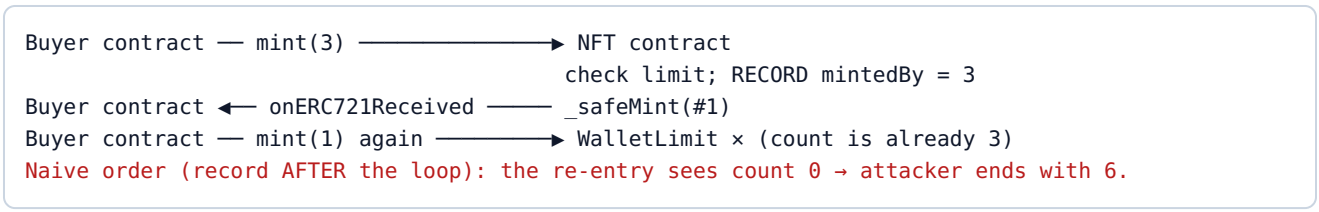


Figure 2 — Re-entrant minting. Same lesson as Day 06: checks → effects → interactions.

Step 5 — Royalties are a question, not a rule

ERC-2981 adds `royaltyInfo(id, salePrice)`, which answers "who should get how much". With 500 basis points (5%), a 2 ETH sale returns 0.1 ETH. Nothing in the token forces a marketplace to pay; a plain transfer pays nothing. Integer maths rounds down: 19 wei × 5% = 0.

Step 6 — ERC-1155: many items, one contract

Each id has its own supply. GOLD (cap 1,000,000) behaves like a coin, SWORD (cap 100) like a limited edition, CROWN (cap 1) like an NFT. `mintBatch` and `safeBatchTransferFrom` move several types in one transaction. There is no unsafe transfer at all, and there is **no per-item approval**: approving a marketplace for GOLD also lets it move the CROWN.

Key code walkthrough

MiniNFT.transferFrom — permission first, then the books

```
address owner = ownerOf(tokenId);            // reverts if the token does not exist
if (msg.sender != owner && _tokenApprovals[tokenId] != msg.sender
    && !isApprovedForAll(owner, msg.sender))
    revert ERC721InsufficientApproval(msg.sender, tokenId);
if (owner != from) revert ERC721IncorrectOwner(from, tokenId, owner); // order fixed (M1)
delete _tokenApprovals[tokenId];              // the old approval must not survive the sale
unchecked { _balances[from] -= 1; _balances[to] += 1; }
_owners[tokenId] = to;
emit Transfer(from, to, tokenId);
```

MiniNFT._checkOnERC721Received — ask before sending

```
if (to.code.length == 0) return;                // a normal wallet can always hold NFTs
try IERC721Receiver(to).onERC721Received(msg.sender, from, tokenId, data) returns (bytes4 r) {
    if (r != IERC721Receiver.onERC721Received.selector) revert ERC721InvalidReceiver(to);
} catch (bytes memory reason) {
    if (reason.length == 0) revert ERC721InvalidReceiver(to); // no hook at all
    assembly { revert(add(32, reason), mload(reason)) }      // pass the receiver's own error on
}
```

LearningNFT.mint — record before you hand over control

```
// checks
if (quantity == 0 || quantity > MAX_PER_WALLET) revert InvalidQuantity(quantity);
if (quantity > MAX_SUPPLY - totalMinted) revert SoldOut(...);
if (mintedBy[msg.sender] + quantity > MAX_PER_WALLET) revert WalletLimit(...);
if (msg.value != MINT_PRICE * quantity) revert WrongPayment(...);   // exact: no silent overpay
// effects  (BEFORE the receiver runs)
uint256 firstId = totalMinted + 1;  totalMinted += quantity;  mintedBy[msg.sender] += quantity;
// interactions
for (...) _safeMint(msg.sender, firstId + i);   // may call msg.sender back
```

GameItems.mintBatch — the same id twice must not dodge the cap

```
for (i ...) { already = 0;
    for (j < i) if (ids[j] == ids[i]) already += amounts[j];   // running total per id
    _checkCap(ids[i], already + amounts[i]); }
_mintBatch(to, ids, amounts, "");      // [CROWN, CROWN] × [1, 1] → ExceedsMaxSupply(2, 1, 1)
```

Evening fix and improvement (LearningNFT)

```
function renounceOwnership() public override onlyOwner {
    uint256 held = address(this).balance;
    if (held != 0) revert FundsStillHeld(held);    // E1: otherwise the ETH is locked forever
    super.renounceOwnership(); }
function freezeMetadata() external onlyOwner { ... metadataFrozen = true; }  // one-way
// setBaseURI now also emits EIP-4906 BatchMetadataUpdate(1, 2^256-1) so marketplaces refresh
```

Results from the run

What	Result
Gas: mint (MiniNFT vs OpenZeppelin)	68,560 vs 68,807
Gas: transferFrom (MiniNFT vs OpenZeppelin)	54,629 vs 55,077
Differential fuzz test (120 random programs)	1,394 steps: 454 succeeded on both, 940 failed on both with the identical error
Re-entrant mint, limit 3	Naive contract: attacker holds **6**. LearningNFT: **3** (3 re-entries tried, all refused)
Royalty on a 2 ETH sale at 500 bps	0.1 ETH to the artist
ERC-1155: 3 item types	One batch 201,004 gas vs 3 single mints 266,127 gas (−24%)
Merkle allowlist (exercise 7)	Writing 1,000 addresses: 22,858,999 gas. Whole Merkle contract: 1,237,697 gas (18.5×)
Quality	117/117 tests · ESLint + Solhint clean · npm audit 0 · Slither: 5 informational (explained) · mutation 38/38 caught

QA & security findings

ID	Sev.	Found by	Finding	Fix
E1	Medium	Evening audit	Owner could `renounceOwnership()` with mint income inside; nobody could ever withdraw it again.	Renounce reverts with `FundsStillHeld` while the balance is not 0.
M1	Low	Shared test suite	MiniNFT checked `from` before permission, so a stranger got a different error than from OpenZeppelin.	Permission checked first.
S1	Low	Slither	Two locals relied on implicit zero.	Initialised explicitly.
L1	Info	Solhint	3 lines of assembly to re-throw the receiver's error.	Kept and justified.

Also attacked and safe: buying from inside the owner's giveaway mint (fresh ids, cap kept), read-only re-entrancy (inside the hook the new owner is already recorded and the old approval is gone), the last-token race (loser refused, keeps their

ETH), royalty receiver 0x0, mint to 0x0, empty ERC-1155 batch. Accepted: royalties are not enforced, operator approval is all-or-nothing, metadata is a link (now freezable), $O(n^2)$ duplicate check in owner-only batches.

Common mistakes

- Updating limits or counters **after** `_safeMint` (re-entrant mint).
- Using `transferFrom` to send to a contract: the token can be stuck forever.
- Accepting `msg.value >= price`: buyers silently overpay and the extra is kept.
- Treating ERC-2981 as enforcement. It is a polite request.
- Storing an `https://` metadata link that the owner can change at any time.
- Clicking "approve all" on an unknown site: the operator gets every NFT you own.
- Letting the owner renounce with funds inside (today's E1).

Five interview questions

1. How is ERC-721 different from ERC-20?
ERC-20 maps an address to an amount of identical units; ERC-721 maps each token id to one owner, so every unit is distinct. ERC-721 also adds per-token approval and safe transfers.

2. Why does `safeTransferFrom` exist, and what risk does it add?
It refuses to send to contracts that cannot handle NFTs, by calling `onERC721Received`. That call gives the receiver control mid-function, so state must be final before it (re-entrancy).

3. What does `supportsInterface(0x80ac58cd)` mean?
ERC-165: "do you implement ERC-721?". The id is the XOR of the 9 ERC-721 function selectors.

4. Are ERC-2981 royalties guaranteed?
No. The contract only reports the amount; the marketplace decides whether to pay. Enforcing it means restricting transfers.

5. When would you choose ERC-1155?
When one project has many token types, some fungible and some unique (game items, tickets with tiers), and wants batch transfers and one deployment.

Exercises summary

Level	Exercise	Key idea
Easy	Rebuild ownership from `Transfer` events; compute the interface id; what `tokenURI` stores	Events are the history; `0x80ac58cd`; a link, not the image
Medium	Soulbound certificate; fix the re-entrant mint	Block transfers in `_update`; record before `_safeMint`
Challenge	Merkle allowlist presale; measure the storage alternative	One 32-byte root, 10-hash proofs, 18.5× cheaper than writing the list

Cheat sheet

ERC-721: `ownerOf`, `balanceOf`, `transferFrom`, `safeTransferFrom` ×2, `approve`, `getApproved`, `setApprovalForAll`, `isApprovedForAll`. Events: `Transfer`, `Approval`, `ApprovalForAll`.
Mint / burn = `Transfer` from / to `0x0`.
Interface ids: ERC-165 `0x01ffc9a7` · ERC-721 `0x80ac58cd` · Metadata `0x5b5e139f` · ERC-2981 `0x2a55205a` · ERC-1155 `0xd9b67a26` · EIP-4906 `0x49064906`.
Receiver hook: return `onERC721Received.selector` (`0x150b7a02`).
Royalty = salePrice × bps / 10,000, rounded down.
ERC-1155: `balanceOf(owner, id)`, `balanceOfBatch`, `safeBatchTransferFrom`, URI template `{id}`, operator approval only.
Rule of the day: any external call (including `_safeMint`) is a door. Check, record, then call.

References

1. EIP-721: Non-Fungible Token Standard — https://eips.ethereum.org/EIPS/eip-721
2. EIP-1155: Multi Token Standard — https://eips.ethereum.org/EIPS/eip-1155
3. EIP-2981: NFT Royalty Standard — https://eips.ethereum.org/EIPS/eip-2981
4. EIP-165: Standard Interface Detection — https://eips.ethereum.org/EIPS/eip-165
5. EIP-4906: Metadata Update Extension — https://eips.ethereum.org/EIPS/eip-4906
6. OpenZeppelin Contracts 5.x: ERC-721 — https://docs.openzeppelin.com/contracts/5.x/erc721
7. OpenZeppelin Contracts 5.x: ERC-1155 — https://docs.openzeppelin.com/contracts/5.x/erc1155
8. ethereum.org: Non-fungible tokens — https://ethereum.org/en/nft/
9. Solidity docs: Security considerations (re-entrancy) — https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
10. Antonopoulos & Wood, *Mastering Ethereum*, ch. 10 (Tokens) — https://github.com/ethereumbook/ethereumbook

DAY 09 / 15 · PHASE 3: SECURITY · 23 OCTOBER 2026

Smart-contract security: break it, then fix it

Five classic bugs, each with a working exploit and a fix. Study guide by Manish Khadka.

1. Learning objectives

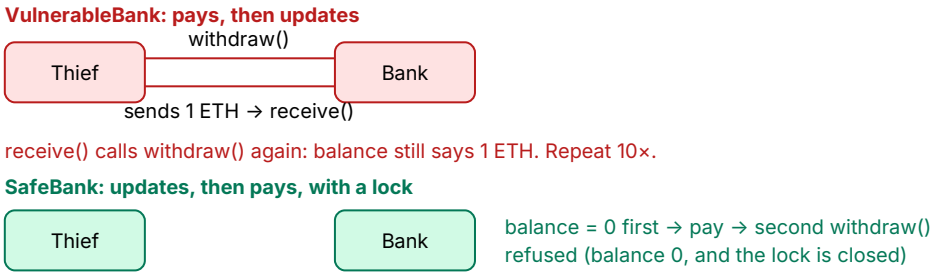
- Explain in plain words why a smart contract must survive **hostile** callers, not just friendly ones.
- Recognise and exploit five bugs: re-entrancy, missing access control, `tx.origin` authentication, front-running and denial of service.
- Apply the fix for each: checks-effects-interactions + lock, `onlyOwner` / `Ownable2Step`, `msg.sender`, commit–reveal, pull payments.
- Write **invariant (fuzz) tests** that find bugs you did not know to look for.

2. The concept, explained simply

A smart contract is a vending machine that holds real money and that **anyone in the world can poke, in any order, as often as they like**. It cannot be patched after deployment. So the question is never "does it work when I use it?" but "what if someone is trying to break it?"

#	Bug	Everyday picture	Fix
1	Re-entrancy	Cashier hands over cash *before* writing it in the book; you ask again while the book is still old.	Update the book first (CEI) + `nonReentrant` lock
2	Missing access control	The "make me owner" button has no lock.	Owner set once in the constructor; `onlyOwner`; two-step hand-over
3	`tx.origin`	Bank checks who started the errand, not who stands at the counter.	Check `msg.sender`
4	Front-running	You shout the quiz answer in a crowded room; someone faster repeats it.	Commit–reveal (sealed envelope)
5	Denial of service	A shop that must refund the last customer first; that customer refuses it.	Pull payments

Step by step: re-entrancy



The thief deposited **1 ETH** and walked away with **10 ETH**; the other 9 ETH belonged to Alice and Bob.

Step by step: front-running

1. Alice sends `solve("satoshi")` with a 2 gwei tip. It waits in the public **mempool**.
2. A bot reads it, copies the call and sends it with a 50 gwei tip.
3. Builders order by fee: the bot's copy runs first and wins 2 ETH; Alice's fails with `AlreadySolved`.
4. **Fix:** Alice first sends `keccak256(her address, answer, salt)` (a sealed envelope), then reveals one block later. The hash includes her address, so a copied reveal does not match the bot's own (non-existent) commitment.

3. Key code walkthrough

The fix for re-entrancy (`SafeBank.withdraw`)

```
function withdraw() external nonReentrant {          // lock: no second entry
    uint256 amount = balanceOf[msg.sender];          // CHECKS
    if (amount == 0) revert NothingToWithdraw();
    balanceOf[msg.sender] = 0;                       // EFFECTS: record first
    emit Withdrawn(msg.sender, amount);
    (bool ok, ) = msg.sender.call{value: amount}("");// INTERACTION: last
    if (!ok) revert TransferFailed();
}
```

The fix for front-running (`CommitRevealPuzzle`)

```
function makeCommitment(address player, string calldata answer, bytes32 salt)
    public pure returns (bytes32) {
    return keccak256(abi.encode(player, answer, salt)); // bound to YOUR address
}
function reveal(string calldata answer, bytes32 salt) external {
    Commitment memory c = commitments[msg.sender];
    if (c.hash == bytes32(0)) revert NoCommitment();
    if (block.number <= c.blockNumber) revert RevealTooEarly(); // at least 1 block later
    if (makeCommitment(msg.sender, answer, salt) != c.hash) revert CommitmentMismatch();
    ...
}
```

The fix for denial of service (`SafeKingOfTheHill.claimThrone`)

```
if (msg.value <= prize) revert BidTooLow(prize, msg.value);
if (king != address(0)) pendingReturns[king] += prize; // RECORD the refund, don't send it
king = msg.sender; prize = msg.value;
```

4. Results from my run

Test	Broken contract	Fixed contract
Re-entrancy (1 ETH deposit; 10 ETH in the bank)	10 re-entries, bank left with 0 ETH, thief profit **10 ETH**	1 entry, 1 refused, bank keeps 10 ETH, profit 0
Cost of safety	withdraw 28,479 gas	32,303 gas (+3,824, about 13%)
Open `initialize()`	Thief becomes owner, wallet emptied	`OwnableUnauthorizedAccount`; wallet keeps 5 ETH
`tx.origin` phishing	Alice clicks "claim"; wallet emptied	Refused (`NotOwner`); 5 ETH stays
Front-running	Bot at position 0 wins 2 ETH; Alice at 1 fails	Bot's copy fails (`NoCommitment`); Alice wins
DoS (refund always fails)	King stuck for ever (`RefundFailed`)	Carol becomes king; the stubborn one only blocks itself

Tests: 31 / 31 passing, including fuzz invariants (200 random programs, 1,257 steps, "ETH held = sum of balances" held after every step).

5. QA & security findings

ID	Sev.	What	Fix
M1	Medium	`SafeWallet` inherited `renounceOwnership()`: one call would lock all ETH for ever.	Override to revert (`RenounceDisabled`)
S1	Low	A `transfer(address,uint256)` without a return value looks like an ERC-20 token (Slither `erc20-interface`).	Renamed `transferBalance` / `pay`
T1	Low	My cross-function re-entrancy test passed for the wrong reason (balance was already 0).	Assert the exact `ReentrancyGuardReentrantCall` error
E1	Low	Evening audit: `transferBalance(bankAddress, x)` parked money where nobody could ever withdraw it.	Revert `InvalidRecipient`

Also: Slither flagged only 2 of my 5 planted bugs (re-entrancy, `tx.origin`). The open `initialize()`, the front-running and the stuck refund were invisible to it: **tools help, exploit tests**

prove. `npm audit`: 0 vulnerabilities. Known issues: weak answers can be guessed offline (K1), a producer can censor a reveal (K2).

6. Common mistakes & 5 interview questions

- Sending ETH before updating your own records.
- Leaving an admin or `initialize()` function open.
- Using `tx.origin` to decide who is allowed.
- Trusting `address(this).balance` as bookkeeping (ETH can be forced in with `selfdestruct`).
- Looping over users or pushing refunds inside someone else's transaction.
- Believing a test that "passes" proves the lock works (my T1).

Q1. What is re-entrancy and how do you stop it? A contract is called again before its first call has finished, while its records are still old. Follow checks-effects-interactions and add a `nonReentrant` lock on every function that moves value.

Q2. Why is `tx.origin` dangerous for authorisation? It is the person who signed the whole transaction, even if a malicious contract is the one calling. A phishing contract the owner calls passes the check. `msg.sender` is the immediate caller and fails for the phisher.

Q3. How does commit–reveal beat front-running? The commitment hides the answer and is bound to the sender's address; the reveal must come in a later block. A bot copying the reveal has no earlier matching commitment of its own.

Q4. Push vs pull payments? Push sends money inside your action, so one refusing receiver can block everyone. Pull records the debt and lets each person collect their own.

Q5. Why use invariant/fuzz tests? An example test only checks cases you thought of. An invariant ("ETH held = sum of balances") checked after random sequences finds bugs you did not know existed; my fuzzer broke a bank with one deleted line in 2 steps.

7. Exercises summary

Level	Task
Easy	Build a commit–reveal commitment by hand in JavaScript and compare with `makeCommitment`; list contracts that look like ERC-20. Why `abi.encode` and not packed encoding?
Medium	Write a pull-payment auction; prove a bidder that cannot receive ETH does not block it (`PullAuction`).
Challenge	Fuzz `LeakyBank` (one deleted line) with the invariant "held = sum of balances"; get the shortest failing program.

Cheat sheet

- Order: **Checks → Effects → Interactions** + `nonReentrant`.
- Every function is public to the world unless locked: `onlyOwner`, `Ownable2Step`, disable `renounceOwnership` when funds are held.
- Authorise with `msg.sender`, never `tx.origin`.
- Mempool is public: hide secrets with **commit–reveal**.
- Never let one receiver block others: **pull, don't push**.
- Keep your own records; do not use `address(this).balance` for accounting.
- Ask "where can money get stuck?" and write an invariant.

8. References

1. Solidity docs, Security considerations: https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
2. SWC Registry: https://swcregistry.io/
3. ethereum.org, Smart contract security: https://ethereum.org/en/developers/docs/smart-contracts/security/
4. ethereum.org, Maximal extractable value (MEV): https://ethereum.org/en/developers/docs/mev/
5. OpenZeppelin Contracts 5.x, Access control: https://docs.openzeppelin.com/contracts/5.x/access-control
6. OpenZeppelin Contracts 5.x, API utilities (ReentrancyGuard): https://docs.openzeppelin.com/contracts/5.x/api/utils
7. Slither static analyzer (Trail of Bits / Crytic): https://github.com/crytic/slither
8. Consensys, Smart Contract Best Practices: https://consensysdiligence.github.io/smart-contract-best-practices/
9. fast-check, property-based testing: https://fast-check.dev/
10. Code for this day: https://github.com/Khadka1996/Learning-Blockchain/tree/main/day-09

Day 10 — DeFi: an exchange with no company behind it

Constant-product AMM (x · y = k) · Sat 24 Oct 2026 · Phase: Smart contracts & DeFi · Learning Blockchain, day 10 of 15

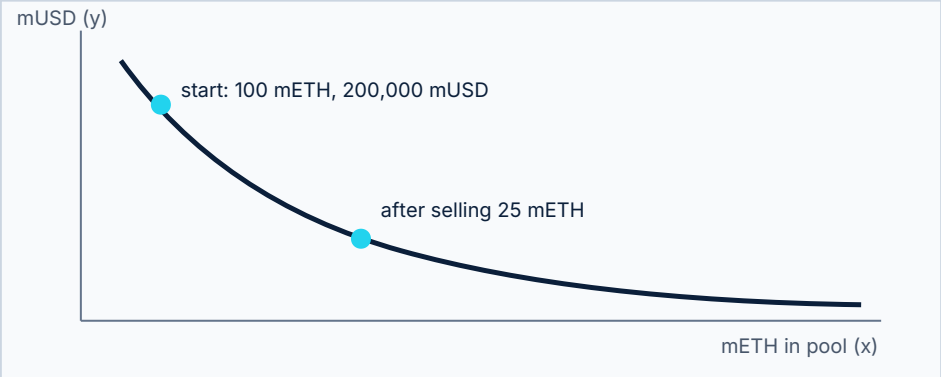
1. Learning objectives

- Explain how a swap pool prices trades using only `x · y = k`.
- Compute output, fee and price impact by hand and in code.
- Explain what liquidity providers earn and risk (fees vs impermanent loss).
- Describe a sandwich attack and why `minAmountOut` limits it.
- Name the safety rules of a pool: own books, count what arrived, locked shares, deadlines, re-entrancy lock.

2. The concept, step by step

The jar. A jar holds 100 apples and 100 oranges (100 × 100 = 10,000). Put in 25 apples: the jar has 125 apples, so it may hand out oranges only down to 10,000 ÷ 125 = 80. You get **20** oranges, not 25. The more you take, the dearer each next orange. Nobody sets the price; the jar does.

1. **Pool.** A contract holds two tokens (reserves `x`, `y`). Anyone may add both and get **LP shares**.
2. **Swap.** Trader sends `Δx`. After a 0.30% fee the contract pays out `Δy = Δx·0.997·y / (x + Δx·0.997)`, rounded **down**.
3. **Fee stays inside.** So `k` grows after each trade and every LP share is worth a bit more.
4. **Price = ratio.** 100 mETH vs 200,000 mUSD means 1 mETH ≈ 2,000 mUSD. Big trades move the ratio (price impact).
5. **Protection.** Every call carries a slippage limit and a deadline.



Every trade slides along the curve; it never reaches zero.

Sandwich attack

Step	Who	What happens
1	Victim	Sends a swap; it waits in the public mempool.
2	Bot	Sees it, buys first with a higher tip. Price moves against the victim.
3	Victim	Swap runs at the worse price.
4	Bot	Sells right after and keeps the difference.

Defence: `minAmountOut`. If the victim says "at least 99.5% of the quote", the bot can squeeze 0.5% at most; ask for more and the trade simply cancels.

3. Key code walkthrough

The price formula (rounds against the trader)

```
function getAmountOut(uint256 amountIn, uint256 reserveIn, uint256 reserveOut)
    public pure returns (uint256) {
  if (amountIn == 0) revert ZeroAmount();
  if (reserveIn == 0 || reserveOut == 0) revert InsufficientLiquidity();
  uint256 amountInWithFee = amountIn * (BPS - FEE_BPS);   // 0.997 as 9970/10000
  return (amountInWithFee * reserveOut)
       / (reserveIn * BPS + amountInWithFee);              // integer division = round DOWN
}
```

Count what really arrived

```
function _pull(IERC20 token, uint256 amount) private returns (uint256) {
  uint256 before = token.balanceOf(address(this));
  token.safeTransferFrom(msg.sender, address(this), amount);
  return token.balanceOf(address(this)) - before;   // fee-on-transfer safe
}
```

Own books, never `balanceOf` for prices

```
uint256 public reserve0;  uint256 public reserve1;   // changed only by add / remove / swap
// swap: received = _pull(...); out = getAmountOut(received, rIn, rOut);
//       if (out == 0 || out < minAmountOut) revert InsufficientOutput(out, minAmountOut);
//       _update(...);  tOut.safeTransfer(to, out);       // effects, then interaction
```

Evening improvement: ask the pool what a deposit will do

```
(uint256 shares, uint256 a0, uint256 a1) = pair.quoteAddLiquidity(d0, d1);
pair.addLiquidity(d0, d1, shares * 995 / 1000, me, deadline);   // minShares from the quote
```

4. Results from the run

Check	Result
Sell 1 / 5 / 50 mETH in a 100 / 200,000 pool	1,974.32 / 9,496.59 / 66,533.20 mUSD (impact 1.28% / 5.03% / 33.47%)
Fees: 10 round trips of 5 mETH	Bob loses 0.2854 mETH; the pool's `k` grows 0.29%; Alice withdraws more than she put in
Sandwich, no limit	Victim loses 29.97%, bot earns 1.4426 mETH
Sandwich, 0.5% limit	Victim loses 0.50%, bot earns 0.0245 mETH
Donation attack	Failed: victim got 10 mETH back
Impermanent loss, price ×2 (exercise 2)	Formula −5.72%, real contract −5.68%
Tests	33 / 33 pass; fuzz 150 programs, 860 steps, 4 invariants held

5. QA & security findings

ID	Sev.	Finding	Outcome
L1	Low	addLiquidity accepted the pool or a token as recipient (shares lost forever)	Fixed (morning)
T1	Low	My fuzz test could not tell expected reverts from bugs	Fixed (morning)
E1–E3	Info	Deadline boundary, emptied pool, pool opened at a silly price	Tested, no defect
E4 / K6	Info	A token that takes balance from the pool makes it insolvent for the last LP	Known issue, documented

No Low, Medium or High issues were found in the evening pass. ESLint, Solhint clean; `npm audit` 0 vulnerabilities; Slither results explained in QA.md.

6. Common mistakes & interview questions

- No `minAmountOut` (or set to 0): a bot decides your price.
- Reading `balanceOf` to price or share: a donation breaks it.
- Rounding in the user's favour: bots repeat it millions of times.
- Using the pool's spot price as an oracle for lending: a flash loan moves it for one block.

Q1. What does x · y = k mean?
After a swap the product of the two reserves must not decrease. It fixes the price curve.

Q2. Why is a big trade worse per token?
It moves along the curve; each extra token costs more (price impact), plus the 0.30% fee.

Q3. How do LPs earn, and what is impermanent loss?
Fees stay in the pool so shares grow. If the outside price moves, arbitrage rebalances the pool and the LP ends with less value than just holding; for a 2× move it is about 5.7%.

Q4. How does a sandwich work and how do you limit it?
A bot buys before and sells after your trade. Set `minAmountOut` (and a deadline); use private mempools for the rest.

Q5. Why lock 1,000 shares on the first deposit?
So the first depositor cannot make one share worth an absurd amount and round later depositors to zero (inflation attack).

7. Exercises summary

Level	Task	Run
Easy	Price-impact table for any pool	`npm run ex1`
Medium	Measure impermanent loss on the real contract and compare with 2√r/(1+r) − 1	`npm run ex2`
Challenge	Exact arbitrage trade size by integer bisection	`npm run ex3`

Cheat sheet
Out = Δx·0.997·y / (x + Δx·0.997) · fee 0.30% stays in pool · price = y / x
IL = 2√r / (1 + r) − 1 (r = price change) · first deposit shares = √(x·y) − 1,000
Always: `minAmountOut`, deadline, own books, count what arrived, round against the user, re-entrancy lock.
Never: use a pool spot price as an oracle; list rebasing tokens.

8. References

1. Uniswap V2 whitepaper — https://docs.uniswap.org/whitepaper.pdf
2. Uniswap V2 docs: how it works — https://docs.uniswap.org/contracts/v2/concepts/protocol-overview/how-uniswap-works
3. Uniswap V2 pair contract — https://github.com/Uniswap/v2-core/blob/master/contracts/UniswapV2Pair.sol
4. ethereum.org, Decentralized finance — https://ethereum.org/en/defi/
5. ethereum.org, Maximal extractable value (MEV) — https://ethereum.org/en/developers/docs/mev/
6. OpenZeppelin Contracts 5.x docs — https://docs.openzeppelin.com/contracts/5.x/
7. Solidity documentation — https://docs.soliditylang.org/
8. Smart Contract Weakness Classification registry — https://swcregistry.io/
9. Angeris et al., "An analysis of Uniswap markets" — https://arxiv.org/abs/1911.03380
10. Slither static analyzer — https://github.com/crytic/slither

DAY 11 / 15 · PHASE 3 — ADVANCED

Gas Optimisation & Upgradeable Contracts

Sunday 25 October 2026 · Manish Khadka · Learning Blockchain series · code: github.com/Khadka1996/Learning-Blockchain/tree/main/day-11

1. Learning objectives

- Explain why **storage writes** dominate gas cost and name the real prices (20,000 / 2,900 / 2,100).
- Apply eight savings (packing, `immutable`, custom errors, `calldata`, cached reads, `unchecked`, one total write) and **prove** behaviour did not change with a differential fuzz.
- Explain a UUPS proxy: `delegatecall`, the ERC-1967 slot, `initialize`, locked implementations.
- Upgrade V1 → V2 without losing a balance, and recognise the classic reordered-storage disaster.

2. The concept, step by step

Part A — Gas is mostly about storage

Every step a contract takes costs **gas**. An addition costs 3. Writing to **storage** — the contract's permanent hard disk made of 32-byte "boxes" (slots) — is the expensive part. Think of a **post office with lockers**: opening a locker costs money whether it holds a letter or a parcel, so use fewer lockers.

Action	Gas
Empty box → filled	20,000
Change a box that already holds something	2,900
First touch of a box in a transaction (cold access)	+2,100
Add two numbers	3

Naive Member: 4 slots
bool · uint256 · uint256 · address

→

Packed Member: 1 slot
address 20 B + uint64 8 B + uint32 4 B = 32 B

Trick	Why it saves gas
Storage packing	One box instead of four; the box is already non-zero, so later writes cost 2,900 not 20,000
`immutable`	Value lives in the code: ~3 gas to read instead of 2,100

Custom errors	4-byte selector instead of a stored string; can carry data
`calldata`	Array read in place, no copy to memory
Cache length / values	Memory reads ~3 gas vs storage 100–2,100
One write per batch	Add up locally, write the total once
`unchecked { ++i; }`	Skip an overflow check that can never trigger

Warning: packing `points` into a `uint32` means it *must* be range-checked. Packing that silently wraps is a bug, not an optimisation.

Part B — Upgradeable contracts (UUPS proxy)

Deployed code can never change. A **proxy** is a shop front with a fixed address that holds the money and records; the **implementation** is the staff (code only). The proxy forwards every call with `delegatecall` — "run your code on *my* storage". Upgrading = pointing the proxy at new staff.



Term	Plain meaning
delegatecall	Borrow another contract's code, run it on your own storage
ERC-1967 slot	Fixed pseudo-random slot where the proxy keeps the implementation address, so it can't collide with variables
UUPS (ERC-1822)	Upgrade function lives in the implementation; cheaper proxy, but every version must keep it or the proxy is frozen
initialize()	Replaces the constructor (proxies never run one); runs once
_disableInitializers()	Locks the bare implementation so nobody can claim it

The classic mistake. V1 keeps `balanceOf` in slot 0 and `totalDeposits` in slot 1. A V2 that declares them in the other order compiles and upgrades happily — then every balance reads 0 while 5 ETH sit in the contract. Only *append* new variables.



3. Key code walkthrough

Packed struct and one-write batch (ClubOptimized)

```
struct Member { address referrer; uint64 joinedAt; uint32 points; } // 32 B = 1 slot
address public immutable owner;                // baked into code, ~3 gas to read

function addPointsBatch(address[] calldata list, uint256 amount) external {
    if (msg.sender != owner) revert NotOwner();
    uint256 len = list.length;                 // cache the length
    for (uint256 i; i < len; ) {
        Member storage m = members[list[i]];   // one storage pointer
        if (m.joinedAt == 0) revert NotMember(list[i]);
        uint256 room = type(uint32).max - uint256(m.points);   // QA E1
        if (amount > room) revert PointsOverflow(list[i]);
        m.points = uint32(uint256(m.points) + amount);
        unchecked { ++i; }                     // i < len: cannot overflow
    }
    totalPoints += len * amount;               // ONE write for the whole batch
}
```

The upgrade gate (PiggyBankV1)

```
constructor() { _disableInitializers(); }                // lock the bare implementation
function initialize(address o) public initializer { __Ownable_init(o); ... }
function _authorizeUpgrade(address) internal override onlyOwner {}  // the ONLY gate
```

Forget `onlyOwner` on `_authorizeUpgrade` and anyone can replace the code. V2 appends `maxPerUser` and `paused` and sets them in `initializeV2` (`reinitializer(2)`, owner only).

4. Results from the run

Action	Naive	Optimized	Saved	%
deploy	613,601	549,965	63,636	10.4%
join (no referrer)	96,580	67,939	28,641	29.7%
join (with referrer)	146,331	78,825	67,506	46.1%
addPointsBatch (10)	299,741	105,684	194,057	64.7%
addPointsBatch (50)	1,382,539	419,580	962,959	69.7%
addPointsBatch again (50)	527,539	419,580	107,959	20.5%
withdraw	30,843	28,743	2,100	6.8%

- Most of the 69.7% is one fact: naive `points` boxes start empty (20,000 gas), the packed box was filled at `join` (2,900).
- The "again" row (every box already non-zero in both) still saves 20.5% from the small tricks.
- `withdraw` saves exactly 2,100 gas: one cold read of `owner` removed.

- Upgrade demo: same proxy address, balances Alice 3 ETH / Bob 2 ETH kept, version 1.0.0 → 2.0.0; Eve's upgrade attempt reverted; the reordered V2 was refused by the storage-layout check.

5. QA & security findings

39 / 39 tests pass; differential fuzz (3 × 200 random programs, ~1,250 steps each) identical on both clubs; ESLint + Solhint clean; `npm audit` 0 vulnerabilities; Slither: nothing new in production code.

ID	Sev.	Finding	Fix
L1	Low	Single-step ownership: a typo could lose upgrade control forever	Ownable2Step
T1	Low	Storage-layout test passed for the wrong reason	Broken V2 given a proper initializer; assert the exact error
E1	Low	Huge reward gave raw Panic(0x11), not typed PointsOverflow	Compare by subtraction (~10 gas/member)
E5	Low	0-value deposit blamed on the cap after the cap was lowered	Check ZeroAmount first
P1–P5, E2–E4	Info	Init front-running, locked implementations, trapped funds, forced ETH, duplicate batch entries, uint32 boundary, partial batch	Verified safe; regression tests

Known issue K1: the owner can upgrade to *any* code — the trust assumption of every upgradeable contract. Production owner = multisig behind a timelock.

6. Common mistakes & interview questions

- Packing small types and forgetting the range check (silent wrap).
- Optimising without measuring; "clever" code that saves 20 gas but adds a bug.
- Reordering, removing or retyping state variables in an upgrade.
- Using a constructor in an upgradeable contract (it never runs on the proxy).
- Leaving the implementation uninitialised, or forgetting `onlyOwner` on `_authorizeUpgrade`.

Q1. Why is a storage write so expensive?
It changes the permanent state every node must keep; 20,000 gas for empty→non-zero, 2,900 for a change, +2,100 for a cold access (EIP-2929).

Q2. How does struct packing work and what is the risk?
Adjacent small fields share one 32-byte slot (address 20 + uint64 8 + uint32 4). Risk: narrow types overflow, so they need explicit range checks.

Q3. What is the difference between a transparent proxy and UUPS?
Transparent proxies hold the upgrade logic in the proxy (bigger, admin checks on every call); UUPS keeps it in the implementation (cheaper), but a version without the upgrade function bricks the proxy.

Q4. Why can't an upgradeable contract use a constructor?
The constructor runs on the implementation's own storage at deploy; the proxy's storage never sees it. Use `initialize` guarded by `initializer`, and lock the implementation.

Q5. What makes a V2 storage-layout compatible?
Every existing variable keeps its slot, name and type; new variables are only appended (or use ERC-7201 namespaced storage). Tools like OpenZeppelin's upgrades plugin check this.

7. Evening exercises

1. **Easy:** `slotsUsed(fields)` — naive member = 4 slots, packed = 1; field order matters.
2. **Medium:** predict the storage gas of rewarding N new members with the real prices; model says 76.0% vs measured 69.7% for N = 50.
3. **Challenge:** `checkUpgrade(old,new)` that flags reorder/removal/retype, confirmed against a real proxy's slot 1.

8. Cheat sheet

Gas: new box 20,000 · change 2,900 · cold +2,100 · add 3.
Savings: pack + range-check · immutable · custom errors · calldata · cache · one total write · unchecked loop counter.
Proxy: proxy = storage + ETH, implementation = code · delegatecall · ERC-1967 slot · initialize once · `_disableInitializers()` · `_authorizeUpgrade` + onlyOwner.
Upgrade rules: append only · never reorder/retype · run the layout check · owner = multisig + timelock.
Measure: same scenario, real receipts, differential fuzz for equal behaviour.

9. References

1. Solidity docs, Layout of state variables in storage — https://docs.soliditylang.org/en/v0.8.24/internals/layout_in_storage.html
2. EIP-2929: Gas cost increases for state access opcodes — https://eips.ethereum.org/EIPS/eip-2929
3. EIP-1967: Proxy storage slots — https://eips.ethereum.org/EIPS/eip-1967
4. EIP-1822: Universal Upgradeable Proxy Standard — https://eips.ethereum.org/EIPS/eip-1822
5. EIP-7201: Namespaced storage layout — https://eips.ethereum.org/EIPS/eip-7201
6. OpenZeppelin, Proxy upgrade pattern — https://docs.openzeppelin.com/upgrades-plugins/proxies
7. OpenZeppelin, Writing upgradeable contracts — https://docs.openzeppelin.com/upgrades-plugins/writing-upgradeable
8. Solidity blog, Custom errors — https://soliditylang.org/blog/2021/04/21/custom-errors/
9. evm.codes, opcode gas reference — https://www.evm.codes/
10. Antonopoulos & Wood, Mastering Ethereum — https://github.com/ethereumbook/ethereumbook

DAY 12 / 15 · Phase: Advanced · 26 October 2026

Oracles, Layer 2 rollups & bridges

Study guide by Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-12

1. Learning objectives

- Explain what an **oracle** is and list the six checks a safe price reader makes.
- Explain how an **optimistic rollup** works and why a fraud proof needs only about $\log_2(n)$ rounds.
- Explain a **lock-and-mint bridge** and the three classic bugs (replay, duplicate signer, unbound message).
- Test defensively: boundaries, bad types, replay, and a differential fuzz against a JavaScript model.

2. The concept, step by step

A. Oracles: getting outside facts onto the chain

A smart contract cannot open a website. An **oracle** writes outside facts (like the ETH/USD price) onto the chain. It is like a weather board at a station: if the board is stale or wrong, everyone who trusts it dresses wrong. A lending app with a wrong price lends too much or force-sells people who were fine. Chainlink feeds publish the median of many nodes, updating when the price moves by a set percent (*deviation*) or after a fixed time (*heartbeat*).

Check in SafePriceReader	Catches
Sequencer up (L2)	Feed frozen while the L2 is down
Grace period	Mass liquidations right after a restart
answer > 0	Zero or negative price
0 < updatedAt ≤ now	Missing or future timestamp
age ≤ heartbeat	Stale price
min ≤ answer ≤ max	Glitch like $1,000,000 per ETH

Every failure **reverts** (fail closed): no price is better than a wrong price.

B. Layer 2 rollups: many transactions, one L1 entry

A rollup runs transactions off-chain and posts them to Ethereum in a **batch** with a claimed **state root** (a fingerprint of all balances). Like a bus instead of 100 taxis: the fixed fare is shared. In an **optimistic** rollup the result is believed unless someone proves fraud within a ~7-day window. The proof is a **bisection game**:

```
sequencer and watcher disagree on the final root of 1,000 txs
round 1: compare root after tx 500   -> differ  -> lie is in 0..500
round 2: compare root after tx 250   -> agree   -> lie is in 250..500
...
round 10: one tx left -> L1 re-runs just that tx -> who lied?
```

The cheating sequencer loses its **bond**; half goes to the watcher, half is burned (so it cannot "catch itself" for profit). ZK rollups instead prove correctness with math (Day 14).

C. Bridges: moving tokens between chains

A **lock-and-mint** bridge locks the token on chain A and mints an IOU on chain B, like a coat check. Validators sign the deposit; the destination mints when at least M of N *different* validators signed. Big hacks: Ronin 2022 (~$600M, 5 of 9 keys stolen), Wormhole 2022 (~$320M, signature check bypassed), Nomad 2022 (~$190M, any message accepted).

Bug	NaiveBridge	SafeBridge
Replay	Signs only (to, amount): reusable	Signs deposit id, marks it processed
Same signer twice	Counts signatures	Signers must be strictly increasing
Cross-chain replay	Nothing binds the chain	Digest has chainid + bridge address
Stolen keys ≥ threshold	Lost	Also lost: that is the trust model

3. Key code walkthrough

SafePriceReader.price(): the checks, in order

```
(, int256 answer, , uint256 updatedAt, ) = feed.latestRoundData();
if (answer <= 0) revert InvalidPrice(answer);            // 0 / negative = broken
if (updatedAt == 0 || updatedAt > block.timestamp)       // never filled, or "from the
future"
    revert InvalidTimestamp(updatedAt);
uint256 age = block.timestamp - updatedAt;
if (age > heartbeat) revert StalePrice(age, heartbeat);  // feed promised an update by
now
if (answer < minAnswer || answer > maxAnswer)           // circuit breaker band
    revert PriceOutOfBounds(answer);
return uint256(answer) * scale;                          // scale to 18 decimals
```

SafeBridge.mint(): count signers, not signatures

```
bytes32 digest = mintDigest(srcChainId_, depositId, to, amount); // binds chain, bridge,
id, to, amount
address last = address(0);
for (uint256 i; i < sigs.length; ++i) {
    address signer = ECDSA.recover(digest, sigs[i]);      // OZ rejects malleable (high-
s) sigs
    if (!isValidator[signer] || signer <= last) revert NotValidatorOrUnsorted(signer);
    last = signer;                                        // strictly increasing = no
duplicates
}
processed[depositId] = true;                              // effects before the external
call
token.mint(to, amount);
```

The evening fix: RollupL1.depositBond

```
const total = (this.bonds.get(sequencer) ?? 0) + (typeof amount === "number" ? amount :
NaN);
if (!Number.isSafeInteger(amount) || amount <= 0 || !Number.isSafeInteger(total))
  throw new Error("BadBond");   // no negative, NaN, 1.5, or text like "5"
```

4. Results from the run

Scenario	Naive	Safe
Feed silent for 2 hours	$2,000	REFUSED: StalePrice(7204, 3600)
Feed reports 0	$0	REFUSED: InvalidPrice(0)
Feed reports -1	CRASH (overflow)	REFUSED: InvalidPrice(-1)
Glitch $1,000,000	$1,000,000	REFUSED: PriceOutOfBounds
L2 sequencer down	$1,800	REFUSED: SequencerDown()

Txs in batch	L1 gas per tx	vs one L1 transfer (21,000)
1	47,128	2.2x more expensive
10	5,738	3.7x cheaper
100	1,597	13.1x cheaper
1000	1,183	17.8x cheaper

Fraud proof: a lie after tx #637 of 1,000 was found in **10 rounds**; the sequencer lost its 10 ETH bond, the watcher got 5 ETH. Bridge: the same 2 signatures replayed 5 times minted **500** tokens on the naive bridge for a 100-token lock; the safe bridge refused with `AlreadyProcessed`. Break-even batch size: **3** transactions.

5. QA & security findings

ID	Sev.	Finding	Fix
L1	Low	Rollup balances near 2^53 lose precision, making a fraud proof impossible	Genesis must be exact safe integers
L2	Low	Zero feed address gave an empty revert	Config checks run first
L3	Low	WrappedToken could be wired to address(0)	Constructor reverts
E1	Low	depositBond accepted -5, NaN, 1.5 and the text "5"	Positive safe integer only (BadBond)
P1–P5, E2–E6	Info	Fee-on-transfer, front-running, replay, self-challenge, 64-byte signatures, band edges, grace edge, Merkle duplication, window edge	All safe; tests added

Final: **56 / 56 tests**, ESLint + Solhint clean, `npm audit --omit=dev` 0 vulnerabilities, Slither 0 high / medium. Known issue: if M validator keys are stolen the bridge loses (trust model).

6. Common mistakes & interview questions

- Using one "stale" time for every feed instead of that feed's heartbeat.
- Signing only (to, amount) in a bridge, with no deposit id or chain.
- Counting signatures instead of different signers.
- Not checking the L2 sequencer feed on Arbitrum or Optimism.
- Trusting JavaScript numbers for money (negative, NaN, text, above 2^53).

1. What is an oracle and why is it a trust decision? It brings off-chain data on-chain; your contract is only as safe as the feed and how defensively you read it.

2. Why does an optimistic rollup take about 7 days to withdraw? That is the challenge window in which anyone can prove fraud. It is the security, not a bug.

3. How does bisection find one bad transaction in 1,000? Halve the disputed range each round by comparing state roots; after $\lceil \log_2 1000 \rceil = 10$ rounds L1 re-runs one transaction.

4. What must a bridge signature bind? Chain id, bridge address, source chain, deposit id, recipient and amount, then mark the deposit processed.

5. Optimistic vs ZK rollup? Optimistic assumes correct and allows fraud proofs; ZK attaches a validity proof, so withdrawals finalize once it is verified.

7. Exercises summary

Level	Task	Run
Easy	Compute $\lceil \log_2 n \rceil$ bisection rounds and check it against real challenges	npm run ex1
Medium	Find the batch size where a rollup beats a plain transfer (formula vs measured: 3)	npm run ex2
Challenge	A JavaScript twin of SafePriceReader, differential-fuzzed against the contract	npm run ex3

Cheat sheet

- Oracle read = sign, timestamp, age, band (+ sequencer on L2). Fail closed.
- Rollup = share one L1 fee; optimistic = fraud proof within ~7 days; ZK = validity proof.
- Bisection rounds = $\lceil \log_2 n \rceil$; L1 re-runs one transaction.
- Bridge = bind everything, mark deposit used, count different signers.
- Wrapped supply must never exceed locked tokens.

8. References

1. Chainlink, Using Data Feeds: https://docs.chain.link/data-feeds/using-data-feeds
2. Chainlink, L2 Sequencer Uptime Feeds: https://docs.chain.link/data-feeds/l2-sequencer-feeds
3. ethereum.org, Layer 2: https://ethereum.org/en/layer-2/
4. ethereum.org, Optimistic rollups: https://ethereum.org/en/developers/docs/scaling/optimistic-rollups/
5. Vitalik Buterin, An incomplete guide to rollups: https://vitalik.eth.limo/general/2021/01/05/rollup.html
6. ethereum.org, Bridges: https://ethereum.org/en/developers/docs/bridges/
7. EIP-4844, Blob transactions: https://eips.ethereum.org/EIPS/eip-4844
8. EIP-2098, Compact signatures: https://eips.ethereum.org/EIPS/eip-2098
9. SWC Registry, Signature malleability (SWC-117): https://swcregistry.io/docs/SWC-117
10. Rekt, Ronin Network: https://rekt.news/ronin-rekt/

Full-stack dApp: web page + wallet + contract

Day 13 of 15 · 27 October 2026 · Phase: Building real applications

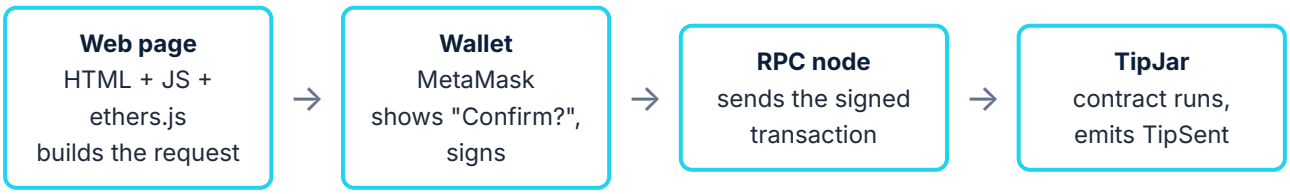
Study guide by Manish Khadka

1. Learning objectives

- Explain the three layers of a dApp (page, wallet, chain) and who holds the private key.
- Build a contract the UI can use safely: bounded reads, clear errors, events.
- Show untrusted on-chain text without XSS; handle wrong network, cancelled signing and stale config.
- Deploy to a local chain or Sepolia while keeping secrets out of git, and test a frontend in a real browser.

2. The concept, step by step

A **dApp** is an ordinary website whose back end is a smart contract. The page never sees a private key: it *asks* the wallet, the wallet *asks you*, and only then a signed transaction goes to the chain.



1. Click *Connect*: the page calls `eth_requestAccounts`; the wallet asks permission and returns the address.
2. The page **reads** (free, instant): `tipCount()`, `getTips(0,20)`.
3. You type an amount and a message. The page checks them **first** (fast, kind errors).
4. The page asks the wallet to send `tip(message)` with value. You confirm; the wallet signs.
5. The page waits for the receipt (about 12 s on Sepolia) and refreshes the list.

	Reading (view)	Writing (transaction)
Gas	No	Yes
Signature	No	Yes (wallet popup)
Speed	Instant	Waits for a block

Network	Chain id	ETH is	Use for
Hardhat local	31337	fake, 10,000 per account	building, tests
Sepolia	11155111	free from a faucet	rehearsal
Mainnet	1	real money	production

3. Key code walkthrough

The contract: bounded, explicit, safe withdrawal

```
function tip(string calldata message) external payable {
    if (msg.value < MIN_TIP) revert TipTooSmall(msg.value, MIN_TIP);   // spam costs money
    uint256 len = bytes(message).length;                               // BYTES, not characters
    if (len == 0) revert EmptyMessage();
    if (len > MAX_MESSAGE_BYTES) revert MessageTooLong(len, 140);
    ...push to _tips, totalTipped += msg.value, emit TipSent(...)
}
function withdraw(address payable to) external onlyOwner {
    uint256 amount = address(this).balance;       // live balance: a re-entrant call finds 0
    if (amount == 0) revert NothingToWithdraw();
    emit Withdrawn(to, amount);                   // effects before the external call
    (bool ok, ) = to.call{value: amount}("");
    if (!ok) revert TransferFailed();
}
```

`getTips(offset, limit)` returns the newest tips first and caps a page at 50, so no read can grow without limit (Day 09 DoS rule).

The page: untrusted text is text

```
el.textContent = tip.message;      // never innerHTML: a stranger wrote this message
// + Content-Security-Policy without 'unsafe-inline'  (second lock)
// + ESLint rule and a test that ban innerHTML in frontend/
```

The secrets check (where the evening audit found bugs)

```
const HARDHAT_PUBLIC_KEYS = Array.from({length: 20}, (_, i) =>
  HDNodeWallet.fromPhrase(PUBLIC_MNEMONIC, undefined, `m/44'/60'/0'/0/${i}
`).privateKey.toLowerCase());
if (BigInt(key) === 0n || BigInt(key) >= CURVE_N) problems.push("not a valid private key");
else if (HARDHAT_PUBLIC_KEYS.includes(key.toLowerCase())) problems.push("PUBLIC test key");
```

4. Results from the run

Measure	Result
Deploy gas (TipJar)	665,179
Tips of 28, 14 and 21 characters	136,722 / 102,390 / 147,360 gas (average 128,824)
Byte sizes of those messages	28 / 17 / 33 bytes (emoji and Nepali text cost more bytes than characters)
Owner withdraw of 0.008 ETH	30,214 gas
Tests	45 passing: 13 contract, 2 fuzz, 13 deploy/env/server, 11 browser E2E, 3 evening audit, 3 exercises

The first tip costs more than the second because it writes a fresh storage slot (`totalTipped` from 0, about 20,000 gas). The third is dearest because its message is longest in bytes.

5. QA and security findings

ID	Sev.	What was wrong	Fix
L1	Low	Two quick account switches left two polling timers	Generation number; older setup stops
L2	Low	A damaged config file was silently replaced	Refuse to overwrite unparseable config
L3	Low	Deploy to the in-memory chain "succeeded" and wrote a dead address	Refuse the `hardhat` network
L4	Low	Page could be framed (clickjacking)	X-Frame-Options and frame-ancestors headers
E1	Low	Only 1 of 20 public Hardhat keys was blocked for Sepolia	All 20 derived and refused
E2	Low	All-zero private key accepted	Valid range 1 to curve order minus 1
E3	Low	"https://" with no host accepted	Parse with `new URL()`, need a host

Re-entrancy on `withdraw`, huge page requests, emoji at the 140-byte limit, path traversal and XSS payloads were all attacked and held. No high or medium findings; Slither shows 6 informational results only.

6. Common mistakes and interview questions

- Putting a private key in the frontend or committing `.env`.

- Rendering chain data with `innerHTML`.
- Counting message length in characters when the contract counts bytes.
- Not handling `chainChanged` / `accountsChanged`, or trusting a stale contract address.

1. **Why can't the page sign by itself?** It never holds the key; the wallet signs so the user approves every transaction.
2. **View call vs transaction?** A view call is a free read answered by a node; a transaction changes state, needs a signature and gas, and waits for a block.
3. **Why validate in both page and contract?** The page gives fast, kind errors; the contract is the real defence because anyone can skip the page.
4. **How do you stop XSS from on-chain text?** Render with `textContent`, add a CSP without inline scripts, and lint against `innerHTML`.
5. **Why paginate `getTips`?** An unbounded array read can exceed gas/RPC limits; a page cap keeps every call bounded.

7. Exercises summary

1. **Easy:** `fits()` and `truncateToFit()`: bytes vs characters, checked against the contract.
2. **Medium:** `fetchAllTips()` using only paged reads (0, 1, 50, 51, 120 tips).
3. **Challenge:** a leaderboard from `TipSent` events whose sum equals `totalTipped()`.

Run with `npm run ex` in `day-13`.

8. Cheat sheet

dApp = page + wallet + contract. Page builds, wallet signs, chain runs.
Reads are free; writes cost gas and wait.
On-chain data is user input: `textContent` + CSP.
Validate twice: page for kindness, contract for safety.
Secrets: `.env` (git-ignored); commit only `.env.example`; block all public test keys.
Handle: chainChanged, accountsChanged, no wallet, Cancel (code 4001), no code at address.
Commands: `npm start` · `npm test` · `npm run node` · `npm run deploy:local` · `npm run check:env`

9. References

- ethers.js v6 docs: https://docs.ethers.org/v6/getting-started/
- EIP-1193 Provider API: https://eips.ethereum.org/EIPS/eip-1193
- MetaMask connect docs: https://docs.metamask.io/wallet/how-to/connect/
- Hardhat deploying guide: https://hardhat.org/hardhat-runner/docs/guides/deploying

- ethereum.org networks: https://ethereum.org/en/developers/docs/networks/
- OWASP XSS prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- Solidity docs: https://docs.soliditylang.org/en/v0.8.24/
- SWC registry (reentrancy SWC-107): https://swcregistry.io/docs/SWC-107

DAY 14 / 15 · PHASE: ADVANCED

Zero-knowledge proofs & smart-contract wallets (ERC-4337)

Study guide · 28 October 2026 · Manish Khadka · github.com/Khadka1996/Learning-Blockchain/tree/main/day-14

1. Learning objectives

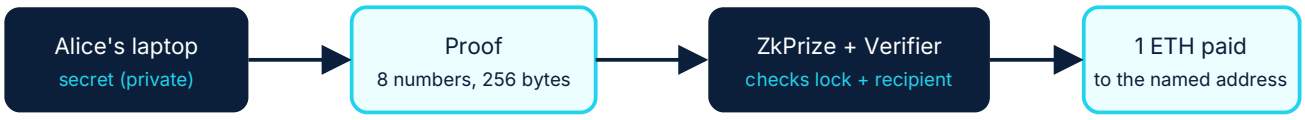
- Explain a zero-knowledge proof in plain words and name its parts: circuit, witness, proof, verifier, public signals.
- Build a Groth16 proof with circom + snarkjs and verify it on-chain to pay out a prize.
- Show why a proof must name its recipient (front-running) and why a weak secret is not protected.
- Explain how ERC-4337 works (UserOperation, bundler, EntryPoint, wallet) and build a wallet with a daily limit and guardian recovery.
- Audit like an attacker: separation of duties, gas as a hole in a spending limit, boundary values.

2. The concept, step by step

Part 1: Zero-knowledge in plain words

A zero-knowledge proof convinces someone that a statement is true **without showing why**. Comparison: you prove you know the Wi-Fi password by connecting; your friend learns that you know it, not the password.

Today's statement: *"I know a secret* `s` *with Poseidon(s) = this public lock, and the prize goes to **this address**."*



Word	Meaning	Comparison
Circuit	The statement as maths rules (constraints). Ours: 416 constraints.	A form whose boxes must add up
Witness	All numbers that fill the form, including the secret. Stays at home.	Your filled-in form
Proof	256 bytes showing the form was filled correctly	A notary's stamp
Verifier	Contract generated by snarkjs that checks the stamp	The bank checking the stamp
Public signals	[lock, recipient]: what everybody sees	The outside of the envelope

Part 2: Account abstraction (ERC-4337)

A normal account (EOA) *is* its key: lose it, money gone; steal it, everything gone at once. A smart wallet is a **contract** with its own rules: cash in your pocket vs a bank account with an ATM limit and a recovery desk. ERC-4337 adds this without changing Ethereum itself.

Piece	Job
UserOperation	A signed "please do this" from the owner (not a transaction)
Bundler	Collects UserOperations, sends one `handleOps` transaction, is refunded for gas
EntryPoint	One official contract: asks the wallet to validate, runs the call, refunds the bundler
Smart wallet	`validateUserOp` (is this signature mine? pay gas) and `execute` (do it, within my rules)

Flow: owner signs → bundler → `EntryPoint.handleOps` → wallet `validateUserOp` → `execute` → daily limit checked → ETH sent.

3. Key code, annotated

```
template ProveSecret() {
    signal input secret;      // PRIVATE
    signal input hash;        // public: the lock
    signal input recipient;   // public: binds the proof to one address
    component h = Poseidon(1);
    h.inputs[0] <== secret;
    hash === h.out;           // rule: Poseidon(secret) must equal the lock
    signal recipientSquare;
    recipientSquare <== recipient * recipient; // keep recipient in a real constraint
}
```

```
uint256[2] memory pub = [secretHash, uint256(uint160(address(recipient)))];
if (!verifier.verifyProof(a, b, c, pub)) revert InvalidProof();
claimed = true;                 // effects BEFORE the external call
(bool ok, ) = recipient.call{value: amount}("");
```

```
function execute(address dest, uint256 value, bytes calldata data) external onlyEntryPointOrOwner
{
    if (dest == address(_entryPoint)) revert ForbiddenTarget(dest); // QA M1
    _spend(value);                                                  // daily limit
    (bool ok, bytes memory ret) = dest.call{value: value}(data);
    if (!ok) revert CallFailed(ret);
}
```

4. Results from the run

What	Result
Proof size / build time	256 bytes (8 numbers) / 642 ms, whatever the secret
Verify + pay out on-chain	258,782 gas
Mallory copies the proof, names herself	rejected: `InvalidProof`
Naive prize (no recipient in proof)	copier wins the 1 ETH without knowing the secret
4-digit PIN as the secret	cracked in 4,822 guesses / 257 ms
Wallet: signed by someone else / same op twice	AA24 signature error / AA25 invalid nonce
Wallet: 0.6 ETH more with 0.5 left	inner call fails (`DailyLimitExceeded`), gas still paid
Guardian recovery after 2 days	new owner works, old key gets AA24
Tests	52 passing (43 contract + 9 exercise)

5. QA & security findings

ID	Sev.	Finding	Fix
M1 (morning)	Medium	Daily limit bypassed through gas: the prefund is paid in `validateUserOp` before `execute`; then `withdrawTo` via `execute` with value 0 pulled ~4.5 ETH out	Cap gas per op at 0.01 ETH; forbid calls to the EntryPoint
L1 (evening)	Low	Owner and guardian could be the same address, so one stolen key could also raise the limit	`OwnerIsGuardian`, `AlreadyOwner` checks
L2 (evening)	Low	A proof naming the prize contract as recipient locked the ETH forever	`RecipientIsPrize` check
K7 (known)	Design limit	"Per day" = per UTC calendar day: 2 ETH left in 35 s around midnight	Not fixed; rolling-window model in exercise 3

Tools: ESLint clean · Solhint clean · `npm audit --omit=dev` 0 vulnerabilities · Slither 9 results, all Info.

6. Common mistakes & interview questions

- Leaving the recipient / `msg.sender` out of the public inputs, so proofs can be stolen from the mempool.
- Using a guessable secret: the lock is public, so anyone can hash guesses.
- One-person trusted setup used with real money; unconstrained public inputs in circuits.

- Forgetting that wallet validation pays gas before any `execute` rule runs.
- Giving one address two roles that were meant to check each other.

1. What does a ZK proof reveal? Only that the statement is true, plus the public signals. Not the secret.

2. Why must the recipient be a public input? A proof is plain data in the mempool. Without the recipient bound into it, anyone can copy it, pay a higher fee and claim the prize.

3. What is a trusted setup? Groth16 needs per-circuit keys made from random "toxic waste"; whoever keeps it can forge proofs. Real projects run a multi-party ceremony: one honest participant is enough.

4. What are the parts of ERC-4337? UserOperation (signed intent), bundler (submits them), EntryPoint (validates and runs them, refunds gas), smart wallet (rules), optional paymaster (pays gas).

5. Why return SIG_VALIDATION_FAILED instead of reverting on a bad signature? Bundlers simulate operations; a revert breaks gas estimation. The EntryPoint then reports AA24 itself.

7. Exercises (evening)

1. **Easy:** brute-force an n-digit PIN from its public lock and estimate the time for 248 random bits (~5e62 years).
2. **Medium:** a sliding-window function that finds the most sent in any window; show the midnight loophole (2 ETH in 60 s).
3. **Challenge:** a `RollingLimit` model: never more than the limit in any 24 h, proven with a property test.

Cheat sheet

- ZK = prove it without showing it. Proof = 256 bytes, verify ≈ 259k gas.
- Public inputs must name who is paid. Secrets need ≥ 128 random bits.
- Groth16 = trusted setup per circuit. Poseidon = hash built for circuits.
- ERC-4337: UserOp → bundler → EntryPoint → `validateUserOp` → `execute`.
- Errors: AA24 = bad signature, AA25 = bad nonce. Validation pays gas first.
- Roles that check each other must be different addresses. Test boundaries (midnight!).

8. References

1. circom documentation: https://docs.circom.io/
2. snarkjs (iden3): https://github.com/iden3/snarkjs
3. ethereum.org, Zero-knowledge proofs: https://ethereum.org/en/zero-knowledge-proofs/
4. Groth, "On the Size of Pairing-based Non-interactive Arguments" (2016): https://eprint.iacr.org/2016/260
5. EIP-4337, Account Abstraction: https://eips.ethereum.org/EIPS/eip-4337
6. ERC-7562, Validation scope rules: https://eips.ethereum.org/EIPS/eip-7562
7. eth-infinitism account-abstraction (EntryPoint v0.7): https://github.com/eth-infinitism/account-abstraction
8. OpenZeppelin Contracts, ECDSA: https://docs.openzeppelin.com/contracts/5.x/api/utils#ECDSA
9. Solidity documentation: https://docs.soliditylang.org/
10. Trail of Bits, zero-knowledge security: https://github.com/trailofbits/publications

Day 15 of 15 · Capstone

A certificate you can check on the blockchain

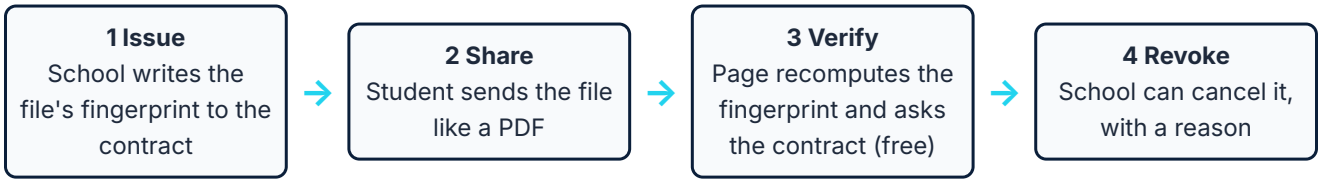
CertCheck: registry contract + web page + self-audit · 29 October 2026 · Phase 4 — Capstone · Manish Khadka

1. Learning objectives

- Combine hashing, signatures, access control, gas saving and a web front-end into one real application.
- Explain why only a **salted fingerprint** (hash) goes on-chain and never personal data.
- Design a verifier that asks "valid in *which* registry, on *which* chain, from *which* issuer?".
- Audit your own code like an outsider: find a bug, write the failing test, fix it, prove it.

2. The idea, step by step

Fake certificates are a real problem; checking one normally means emailing the school and waiting. CertCheck works like a **public notice board that nobody can secretly edit**.



Idea	Plain words
Fingerprint (keccak256 hash)	Same file → same code. One changed letter → a completely different code (Day 1).
Salt	16 random bytes inside the file. Without it, anyone could guess "Manish Khadka, Blockchain 101", hash it and confirm it. With it there are 2^{128} guesses per try.
Canonical JSON	One exact way to write the file (sorted keys, no spaces, normalised Unicode) so the same certificate always gives the same hash.
Per-issuer records	Each school has its own column `(issuer, hash)`, so one school can never take another's certificate (finding M1).
Registry + chain inside the file	A forger's copy of the contract is refused (finding M2).

Who can do what

	Issue	Revoke	Verify
Admin	only after making itself an issuer (visible)	any	yes
Issuer (school)	yes	its own, while still an issuer	yes
Anyone else	no	no	yes, free, no wallet

3. Key code, annotated

Storing a fingerprint (contract)

```
mapping(address issuer => mapping(bytes32 certHash => Record)) private _records;
struct Record { uint40 issuedAt; uint40 revokedAt; }   // 10 bytes = ONE storage slot

function _issue(bytes32 certHash) private {
    if (certHash == bytes32(0)) revert ZeroHash();
    Record storage r = _records[msg.sender][certHash]; // key includes the issuer (M1 fix)
    if (r.issuedAt != 0) revert AlreadyIssued(certHash); // a revoked record stays forever
    r.issuedAt = uint40(block.timestamp);
    emit CertificateIssued(msg.sender, certHash);
}
```

Revoking safely (finding M3)

```
bool ownActiveIssuer = msg.sender == issuer && hasRole(ISSUER_ROLE, msg.sender);
if (!ownActiveIssuer && !hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) revert
NotAllowedToRevoke(msg.sender);
// a removed issuer (stolen key) can no longer cancel real certificates
```

Never lose the last admin (evening finding L2)

```
function _revokeRole(bytes32 role, address account) internal override returns (bool revoked) {
    if (role == DEFAULT_ADMIN_ROLE && hasRole(role, account) && adminCount == 1) revert
LastAdmin();
    revoked = super._revokeRole(role, account);
    if (revoked && role == DEFAULT_ADMIN_ROLE) --adminCount;
}
```

The verifier (off-chain)

```
cert = Certificate.parse(fileText);                // BAD_FILE if anything is odd
if (cert.registry != expected.address || cert.chainId != expected.chainId)
    return { verdict: "WRONG_REGISTRY" };           // M2
const r = await registry.verify(cert.issuer, Certificate.hash(cert)); // free read
// → VALID · VALID_OLD_ISSUER · REVOKED · NOT_FOUND
```

4. Results from the run

```
gas: deploy registry              755,992
gas: issue 1 certificate          70,391
gas: issue a class of 30 (batch)  766,567 (25,552 each)
gas: revoke                       56,087
verdict (genuine file):           VALID (free, a few milliseconds, no transaction)
verdict (one letter changed):     NOT_FOUND
verdict (after revoke):           REVOKED
non-issuer tries to issue:        rejected — AccessControlUnauthorizedAccount
bytes of personal data on-chain:  0
```

Batch issuing is about 2.8× cheaper per certificate than one by one (25,552 vs 70,391 gas).

5. QA & security findings

ID	Sev.	What went wrong	Fix
M1	Medium	First design stored certificates by hash only. A rival issuer could copy the pending transaction, pay more fee and become the "issuer".	Records per (issuer, hash)
M2	Medium	A forger deploys a copy of the contract and "issues" anything there.	File names registry + chain; verifier refuses others
M3	Medium	A removed issuer (stolen key) could still cancel real certificates.	Only an active issuer or the admin may revoke
L1	Low	Form default date used UTC: yesterday in Nepal before 05:45.	Use local date
L2	Low	**Evening:** the only admin could renounce itself → no admin ever again.	`adminCount` + `LastAdmin`

Tools: 46 tests passing (unit, fuzz, attacks, browser end-to-end, deploy, audit, exercises) · ESLint + Solhint clean · `npm audit`: 0 vulnerabilities · Slither: 3 triaged false positives (the `== 0` "never happened" markers). Known issues: one admin key (use a multisig), certificates issued by a thief before removal, address ≠ real school name, look-alike letters.

6. Common mistakes

- Putting names or grades on a public chain. Put a **salted hash** instead.
- Checking only "is this hash valid somewhere?" and not *registry, chain and issuer*.
- Forgetting that removing a role must remove **all** its powers.
- Letting one key be both owner and only admin. Plan the hand-over.
- Trusting a self-audit as a stamp. Get an independent review before real use.

7. Five interview questions

1. Why store a hash and not the certificate? Privacy and cost: the chain is public forever and storage is expensive. A hash proves the file existed unchanged without revealing it.

2. Why a salt? Hashes of guessable data can be brute-forced. 16 random bytes make each guess one of 2^{128}.

3. What is front-running and how did it hit v1? Anyone can read pending transactions and send a copy with a higher fee. In v1 a rival could take over a hash; per-issuer storage makes the copy harmless.

4. Why must the file contain the registry address and chain id? Otherwise an attacker deploys their own contract, issues the same hash there, and a naive verifier says "valid".

5. What is canonical JSON? A single agreed byte representation (sorted keys, no whitespace, Unicode NFC) so equal data always produces the same hash. See RFC 8785.

8. Exercises (tonight)

1. **Easy:** count how many of 256 fingerprint bits change after one letter (about half) and show reordered keys change 0.
2. **Medium:** turn a verdict into one plain sentence using a vetted issuer-name list (case-insensitive addresses; unknown issuer is flagged).
3. **Challenge:** verify a folder of files at once and flag two different files that reuse the same certificate id.

Cheat sheet
Hash = fingerprint · Salt = random bytes so guessing fails · Canonical = one exact text · Store per (issuer, hash) · Verify = registry + chain + issuer + status · Revoke = active issuer or admin · Never remove the last admin · Admin = multisig in production · Audit: failing test first, then the fix.

9. References

1. OpenZeppelin Contracts — Access Control: https://docs.openzeppelin.com/contracts/5.x/access-control
2. Solidity docs — Layout of state variables in storage: https://docs.soliditylang.org/en/v0.8.24/internals/layout_in_storage.html
3. Ethereum.org — Maximal extractable value (MEV): https://ethereum.org/en/developers/docs/mev/
4. RFC 8785 — JSON Canonicalization Scheme: https://www.rfc-editor.org/rfc/rfc8785
5. Blockcerts — open standard for blockchain credentials: https://www.blockcerts.org/guide/
6. Slither static analyzer: https://github.com/crytic/slither
7. Smart Contract Weakness Classification registry: https://swcregistry.io/
8. Consensys — Smart contract best practices: https://consensys.github.io/smart-contract-best-practices/
9. Antonopoulos & Wood, Mastering Ethereum: https://github.com/ethereumbook/ethereumbook

WRAP-UP

What I learned in 15 days

From one hash on Day 1 to a working certificate-checking app on Day 15

The big picture

Day by day, I built a mineable chain, signed transactions, Merkle proofs, a mini-EVM, tested Solidity contracts (a vault, an ERC-20 token, NFTs, an AMM, upgradeable proxies, an oracle reader, a bridge), a full-stack dApp, a zero-knowledge proof, an ERC-4337 smart wallet and the CertCheck capstone. Every day has runnable code, tests, a QA and security review, exercises and a study guide.

5 lessons

1. **A hash is a fingerprint, and that idea carries everything.** Blocks, Merkle trees, signatures, certificates and zero-knowledge locks all start from it (Days 1, 2, 4, 14, 15).
2. **Blockchain code is hard to fix after launch, so test like an attacker before it ships.** Re-entrancy, front-running, bad oracles and bridge replays all appeared in my own code until a test caught them (Days 6, 9, 10, 12).
3. **Everything public can be copied.** A transaction in the mempool, a proof, the hash of a guessable secret: design so that a copy cannot hurt you (Days 9, 14, 15).
4. **Ask who is allowed to do what, and who can take it back.** Most bugs I found were about roles: a removed issuer who could still revoke, a guardian who was also the owner, a registry left with no admin (Days 9, 14, 15).
5. **A passing test is only proof of what it checks.** Reviewing a second time, as a stranger, found a bug every single day. Write the failing test first, then the fix.

Next steps

- Deploy CertCheck to the Sepolia testnet with a multisig admin.
- Read a real professional audit report from start to finish.
- Learn Foundry for fuzzing and invariant testing.
- Write a circom circuit that does something genuinely useful.
- Get an independent human review before any real money or real certificates depend on the code.

Glossary

Term	Plain meaning	Day
Hash (SHA-256, keccak256)	A fixed-size fingerprint of any data. Change one bit of the input and the fingerprint changes completely.	1
Block	A batch of transactions plus a header that includes the hash of the previous block.	2
Proof of Work	Trying nonces until a block's hash is below a target, so making blocks costs real effort.	2
Private / public key	The private key signs; the public key (and the address made from it) lets anyone check the signature.	3
Digital signature	Proof that the owner of a key approved exactly this message, and that it was not changed.	3
Nonce (account)	A counter of transactions sent; each one must use the next number, which stops replays.	3, 5
Merkle tree / proof	Pairs of hashes hashed up to one root; a short path of hashes proves one item is included.	4
Fork / consensus	Two competing chains; nodes agree to follow the one with the most total work.	4
EVM	The small stack computer every Ethereum node runs, so all get the same result.	5
Gas	The price of each step of work. Fee = gas used × gas price.	5
Storage slot	One of 2^{256} 32-byte lockers a contract owns.	5
Smart contract	A program stored on the chain that holds state and runs when called.	6
Re-entrancy	An attacker's contract calls back in before the first call has finished updating balances.	6, 9
ERC-20	The standard interface for fungible tokens (balances, transfer, allowances).	7
Allowance	Permission for another address to spend up to a set amount of your tokens.	7
ERC-721 / ERC-1155	Standards for unique NFTs, and for many token types in one contract.	8
ERC-2981	A standard way for an NFT to say how much royalty to pay, and to whom.	8
Front-running	Someone sees your pending transaction and gets theirs in first to profit from it.	9, 10
AMM (x·y=k)	A pool that prices a swap by keeping the product of its two reserves constant.	10
Slippage limit	The worst price you accept; it protects you from sandwich attacks.	10
Proxy (UUPS)	A contract that keeps the data and forwards calls to replaceable logic, so code can be upgraded.	11

Oracle	A feed that brings outside data (like a price) onto the chain; must be checked for staleness.	12
Rollup (Layer 2)	Runs transactions off the main chain and posts results back; optimistic rollups use fraud proofs.	12
Bridge	Moves tokens between chains; a frequent target of replay and signer attacks.	12
dApp	A web app whose back end is a smart contract, used through a wallet such as MetaMask.	13
Zero-knowledge proof	Proving you know something (like a secret) without revealing it.	14
Account abstraction (ERC-4337)	Smart-contract wallets with custom rules: daily limits, guardians, recovery.	14
Audit	A structured security review that rates findings by severity and proves every fix with a test.	15

Repository

All code, tests, QA reports and exercises: **github.com/Khadka1996/Learning-Blockchain** — one folder per day (day-01 … day-15).

Author: **Manish Khadka** · khadka-manish.com.np · linkedin.com/in/manishkhadka1996